Capability or Optimizer? What Lets an LLM Proposer Leave Its Template Family on Circle Packing

Anonymous authors
Paper under double-blind review

Abstract

Zero-shot LLM proposers on circle packing concentrate on a grid-plus-filler family whose best value follows in closed form from N. We ask what lets a proposer leave it. A fixed SLSQP program with no model in the loop clears the family in 45 of 45 runs and beats the best model-written program at every cell: the optimizer does the clearing. That clearing is measured at $N \leq 31$ within a 120-second wall: budget-matched at $N = 57$ and 59 the same program clears 28 of 28 valid outputs, of 30 launched, and at $N = 73$ it cannot complete one restart in that wall, a limit of the budget, not of the optimizer. Across two proposer tiers and three output channels, the tiers differ in whether their programs drive that optimizer past the family. A Sonnet-tier program with `numpy` and `scipy` clears in 23 of 25 valid programs, or 23 of 41 once every invocation is charged. Without the libraries, on the same path and budget, it clears none. A weak-tier program with the libraries clears none of its 90 invocations, and 4 of 90 when its unreadable output is re-read leniently. Handed the better construction, the weak tier chooses it but builds it in 6 of 14 valid outputs: execution, not preference, is the bottleneck. The template is contingent on the serving path and the request parameters it carries, the reasoning setting included: it is neither confined to one instrument nor invariant across them, and a companion paper measures where it holds and where it breaks. Twenty-one registered arms are reported here, whichever way each went; six more are reported in a companion paper.

1 Introduction

Ask a language model to place 21 circles in a unit square to maximize the sum of their radii, and it will not search. It reaches for the nearest square grid, five by five ($k = 5$) at radius $1/(2k)$, and truncates it to 21 circles, leaving four cells empty. A better construction is one parameter away: a four-by-four grid with interior-vertex fillers of radius $(\sqrt{2} - 1)/(2k)$, one in each gap between four grid circles. The grid-plus-filler family has a closed-form best value at every N; this paper asks what it takes for a proposer to leave it.

Circle packing is the showcase benchmark of LLM-driven discovery, which FunSearch (Romera-Paredes et al., 2024) opened. AlphaEvolve (Novikov et al., 2025) reported 2.63586276 at $N = 26$. Later systems cluster: ShinkaEvolve (Lange et al., 2025) 2.6359831, or 2.6359777 strict; HELIX 2.63598308 (Su et al., 2026); GigaEvo 2.63598 (Khrulkov et al., 2025); AdaEvolve 2.636 (Cemri et al., 2026); and ThetaEvolve (Wang et al., 2025) claims new best-known bounds. Every one pairs a capable model with an executed optimizer; two critiques from outside ask what each contributes: Gideoni et al. (2026) show simple baselines recover much of the reported advantage, and Berthold et al. (2026) show classical solvers do too. We ask the same question from inside, varying the two factors separately.

The result is the optimizer; the table locates it. Two tiers of one vendor, weak and Sonnet, cross three channels: direct coordinate emission, a program allowed only `math`, and a program allowed `numpy` and `scipy`. Table 1 scores every cell under one clearance rule. Every prediction whose verdict this section reports was registered before the arm's first scored run, with the arm-S exception Table 5 records; readings marked post hoc were not. Commits, amendments and outcomes are in the deviation table, Supplement S10. One row clears at rate, in the sense §2.4 defines. A Sonnet-tier program with the libraries clears the family in

23 of 25 valid programs, at 3 of 3 cells (§3.5); its margins over the argmax are in its rows of Table 3. A weak-tier program with the same libraries clears 0 of 19 valid over 90 invocations, and its 95% Wilson upper bound sits under the registered 20% bar. A post hoc lenient re-read of its unreadable outputs recovers 4 clearing programs, 4 of 90 invocations. A Sonnet-tier program without the libraries clears 0 of 37 on the agent runtime and 0 of 35 on the library arm's (arm CL) own path and budget. The library's credit is that same-path gap, 0 of 35 against 23 of 25 (the isolating arm, arm CCP). Two cells show low-rate escapes, both out-of-family constructions: one Sonnet direct emission and two weak-tier emissions at one extend cell, from a post hoc sweep, carried by the direct-emission rows of Table 1 and their note. A fixed random-restart SLSQP program with no model in the loop clears 45 of 45 through the same pipeline and beats the best Sonnet program at every cell (the optimizer-alone baseline, arm B). The optimizer clears the family: the Sonnet tier's programs clear because they invoke it, and the weak tier's programs invoke it and do not clear. That clearing, 50 restarts inside a 120-second wall, is measured at $N \leq 31$: budget-matched at $N = 57$ and 59 the same program still clears 28 of 28 valid outputs, of 30 launched, and at $N = 73$ it cannot complete one restart in that wall, a limit of the budget, not of the optimizer (arm B2, §3.5).

Why: execution, not preference. Handed the better in-family construction, the weak tier builds it correctly in 6 of 14 valid outputs (the argmax-in-hand arm, arm CH). A post hoc attempt-level reading finds it reaching for that construction in 36 of 45 invocations. In the math-only channel the weak tier reaches the argmax construction exactly twice in 78 valid programs and never passes it. A second weak-tier model family, gemma-4-26b at a second vendor (arm GM3, reported in the companion paper), emits the argmax construction at 27 of 43 valid outputs at the cells where template and argmax differ; the original weak tier emits it in 2 of 57. At a held-out cell the original selects the right template and mis-places its one filler. The template is what the model builds reliably, not what it prefers; the library optimizer supplies the execution the weak tier lacks.

And a methods result the table needed. The weak tier's template is a closed form: $k^*(N) = \text{round}(\sqrt{N})$ names the grid. Its value is the empirical modal output at all seven tested N and at four of five held-out N, listed in Supplement S9. Built from zero-shot calls, it is contingent on the setting it was measured in. The same prompt sent to the vendor's pinned API yields 0 valid packings in 105, so every prediction there was unscoreable (the pinned-path arm, arm P). The agent runtime yields the template in 4 of 6 in a post hoc control, and extended thinking restores validity and the template (the thinking-budget arm, arm P-D). A study that reasons about a zero-shot characterization in a loop owes that step a measurement; six further arms measure it under conditioning, iteration and a change of vendor, and a companion paper reports them.

Contributions.

1. *The optimizer clears the family; the tiers differ in whether their programs drive it past the family.* A fixed SLSQP program with no model in the loop clears 45 of 45 and beats the best model-written program at every cell, at $N \leq 31$ within a 120-second wall. Among model-written programs, only a Sonnet-tier program with `numpy/scipy` clears the family's best expressible value at rate: 23 of 25 valid programs, or 23 of 41 once every invocation is charged (four refusals are charged to the serving path). The isolating arm attributes the gap to the library. Every weak-tier trap cell reads zero under the registered scoring; the post hoc lenient reading recovers 4 of 90 invocations.

2. *The bottleneck is execution.* Handed the argmax, the weak tier builds it in 6 of 14 valid outputs. In code the argmax is reached and never passed; at a second vendor, the model family that can execute the harder construction emits it; at a held-out cell, the right template gets the wrong filler.

3. *The characterization depends on the instrument.* The template is contingent on the serving path and the request parameters it carries, the reasoning setting included: it is neither confined to one instrument nor invariant across them, and a companion paper measures where it holds and where it breaks.

Scope. Two tiers of one vendor; a third alias reached 13% validity; Supplement S3 reports it. Three trap cells, $N = 13$, 21 and 31, carry every channel, and the direct weak-tier row adds $N = 43$. One clearance rule, stated in the notation subsection, carries every clearance verdict. The two serving paths, agent runtime

Tier	Channel	Instrument	Cells	Clear	Note
weak	direct emission	agent runtime	13, 21, 31, 43	0 of 57	trap cells; at the extend cell $N = 20$ (arm M) 2 of 15 clear, both out-of-family
weak	direct emission	pinned API	7 square cells	0 valid of 105	unscoreable (arm P)
Sonnet	direct emission	agent runtime	13, 21, 31	1 of 30	one out-of-family packing at $N = 31$; 27 of 30 valid at 10^{-9}, the clearance among them (arm S)
weak	math-only program	agent runtime	13, 21, 31	0 of 78	reaches the argmax exactly 2 times, never passes it (CC, CC2)
weak	math-only program	pinned API	13, 21, 31	0 of 12	two cells under the five-valid floor; 0 of 6 at the scoreable cell (arm P code cells)
Sonnet	math-only program	agent runtime	13, 21, 31	0 of 37	annealing and multi-restart search, still 0 (CCS)
Sonnet	math-only program	pinned API	13, 21, 31	0 of 35	the library row's path and 120 s budget without the libraries; best per cell 1.625, 2.1, 2.673, none at the argmax (CCP)
weak	numpy/scipy program	pinned API	13, 21, 31	0 of 11; 0 of 19 pooled	arm CL, then pooled with its registered top-up CL-W (90 invocations; the $N = 21$ cell stays under the five-valid floor): Wilson upper bound 16.8%, under the 20% bar; a post-hoc reparse of the `numpy`-scalar stdouts reads 4 of 41 (CL, CL-W)
Sonnet	numpy/scipy program	pinned API	13, 21, 31	**23 of 25**	3 of 3 cells; excess over the argmax 0.67–4.24%, median 3.62% (CL)
none	numpy/scipy program, fixed	local, 120 s	13, 21, 31	**45 of 45**	a fixed random-restart SLSQP reference program, no model, through the library row's pipeline; 3 of 3 cells at the 20% bar; best per cell exceeds the Sonnet best at 3 of 3 cells (arm B)

Table 1: **Who clears the family argmax.** Each row is one arm or one registered pair of arms, scored under §2.4's clearance rule: valid at 10^{-9} and above the family argmax by more than 10^{-6}, the argmax recomputed in closed form. "Clear" counts valid outputs. Instrument names the serving path, which arm P shows is not interchangeable: the same weak-tier prompt yields 35 of 45 valid packings (78%) through the agent runtime and 0 of 105 through the pinned API. Rows are not pooled across instruments.

and pinned API, are never pooled. The Sonnet contrast runs on one serving path and one budget as of the isolating arm (arm CCP); the weak tier's math-only cells ran at 10 seconds and its library cells at 120, as the limitations section discloses. The direct rows are unconditioned calls; the anchor result is scoped to them.

What is not claimed. Nothing about mechanism inside the weights: the model *emits*, the distribution *concentrates*, the closed form *identifies the modal output*, and the argmax-in-hand arm probes the leading mechanism-free alternative. Nothing about what any named system does at generation zero, or whether its diversity machinery (Mouret & Clune, 2015; Lehman & Stanley, 2011) repairs or explores. Preregistration is an adopted standard (Thomas et al., 2026; Williams et al., 2026; Vaccaro, 2026; Zhang, 2026), not a contribution. Behavioral anchoring is established territory (Huang et al., 2026; Lou & Sun, 2024; Malberg et al., 2025); the anchor here is a construction template, not a number.

2 The family and its argmax

2.1 The benchmark

Maximize the sum of radii of N non-overlapping circles in the unit square. Feasibility is decidable exactly from emitted coordinates; scoring needs no human. The direct-emission prompt (Supplement S11) fixes the count, states containment and non-overlap, forbids writing or executing code, and demands a raw Python list of `[x, y, r]` triples. The two program channels replace the code-free clause with a program contract, `math` only or `math` with `numpy` and `scipy`, and score the printed output the same way.

2.2 The recipe family

Three terms: the **recipe family** is the parametric set of grid-plus-filler constructions; a **template** is one member; the **anchor** is the template the distribution concentrates on at a given N.

A $k \times k$ grid places one circle per cell center with $r_{\text{grid}} = 1/(2k)$; m fillers sit on interior grid vertices, tangent to their four surrounding grid circles, with $r_{\text{filler}} = (\sqrt{2}-1)/(2k)$, $0 \le m \le (k-1)^2$. Hence $V(k,m) = k/2 + m(\sqrt{2}-1)/(2k)$. When $N < k^2$ the model does not drop to a smaller grid and fill it; it *truncates*, laying out the $k \times k$ lattice, occupying N cells and keeping the radius, giving $T(k,N) = N/(2k)$. These reproduce, with no fitting, every anchor value the study's earlier runs had recorded: $V(5,1) = 2.5414214$, $V(5,2) = 2.5828427$, $V(5,0) = 2.500$, $T(5,23) = 2.300$, $V(4,7) = 2.3624369$.

The **family argmax** at N is the largest value the family can express there, recomputed in closed form over every admissible (k,m). Inside a trap zone it is the larger of $V(k^*{-}1, N-(k^*{-}1)^2)$, the drop-and-fill construction, and the template anchor $T(k^*,N)$. An independent linear program, blind to the recipe, recomputes every closed-form value from the coordinates and aborts on disagreement. It covers 83 configurations at $k = 2\ldots7$ with drift below 10^{-9}, and a further 130 at $k = 8$ and 9 with worst drift 1.8×10^{-14}; Supplement S9.1 lists them. Across the 155 weak-tier rows collected before the extend-cell arm (arm M), only two exceeded the family best, both below the 10^{-6} bar; the clearance rule of §2.4 records their excesses.

2.3 The order rule is a definition; the branch rule is the hypothesis

We write $k^*(N) = \text{round}(\sqrt{N})$ for the nearest-square order. This is a *definition*, not a hypothesis: two formalizations of "nearest" agree on the integers. The falsifiable content is the **branch rule**: $k^{*2} \le N \to$ extend with $m = N - k^{*2}$ fillers (converge); $k^{*2} > N \to$ truncate the k^*-grid (trap). The extend branch survived testing only at $m \le 1$. A registered extension disconfirmed it at $m \ge 4$, where the model prefers the $(k^*{+}1)$ rectangle grid; Supplement S4 has the modal outputs; the deviation table has the verdict. Substituting k^* into the branch condition gives the *trap zones* $N \in [k^2-k+1, k^2-1]$: $[13,15]$, $[21,24]$, $[31,35]$, $[43,48]$, $[57,63]$.

At a converge cell the extend prediction *is* the argmax: we exhausted $N = 10\ldots80$ and found no exception. The sweep, `extend_branch_scope.py`, finds 38 converge cells, none discriminating, and 27 of 33 trap cells discriminating. So every discriminating cell is a trap cell, and the surviving in-family prediction is the truncate branch $T(k^*,N) = N/(2k^*)$. Every arm tests truncate against drop-and-fill; the governing quantity is the signed distance $N - k^{*2}$.

2.4 Notation and conventions

Terms used throughout:

Term	Meaning
$k^*(N)$	nearest-square grid order, round($\sqrt{N}$) (a definition, §2.3)
$V(k,m)$, $T(k,N)$	extend-with-m-fillers value $k/2 + m(\sqrt{2}-1)/(2k)$; truncated-grid value $N/(2k)$
anchor	the template the proposal distribution concentrates on at a given N
on-prediction	emitted sum within 2×10^{-3} of the registered predicted value
rival (argmax)	the highest-valued member of the recipe family at that N when it differs from the prediction: inside a trap zone, $V(k^*-1, m)$
discriminating cell	N where prediction $\neq$ family argmax; only these can distinguish anchoring from family search
validity	non-overlap and containment at 10^{-6} (primary) and 10^{-9} (logged), both always reported
trap / converge	branch by sign of $N - k^{*2}$: truncate when $k^{*2} > N$, extend when $k^{*2} \leq N$
tier	nominal capability class of the serving alias addressed (weak/mid/top): here the weak tier is the vendor's Haiku alias (`claude-haiku-4.5`) and the Sonnet tier its Sonnet alias (`claude-sonnet-4.5`); `opus_alias` is a bare alias with no attestable weights binding
instrument	the serving path: the agent runtime (decoding parameters not exposed, conditioning context locked only in part) or the pinned API, the vendor's chat-completions endpoint reached through OpenRouter (temperature 1.0, no system prompt); never pooled
UNSCOREABLE	cell with fewer valid samples than its arm's registered floor; claims nothing
evaluability floor	the minimum valid-sample count a cell needs before its verdict counts, fixed per arm at that arm's registration and *not* common across arms: 3 for the second-vendor chain (arms GM, GM2 and GM3, companion paper), 5 for arms CN, CP, P, CL, CL-W, CCP, P-D and B2, none for arm CH. Where the loosest one carries a verdict we report the stricter reading beside it
family argmax	the highest value the recipe family can express at N, recomputed in closed form; every clearance verdict is measured against it, and it is the number a discovery claim from a measured proposer has to beat. Table 1 says which proposers clear it
clears at rate	a cell clears *at rate* when its clearance count meets the arm's registered rate bar (20% of valid outputs for arms CL, CCS, CL-W, CCP and B; arm B2 registered none) at every scoreable cell; in plain words, 20% or more of valid outputs clear, at every scoreable cell. A cell with one or two clearances out of dozens shows an *escape*, counted and named, not a clearance at rate
five-valid floor	the evaluability floor of 5, the one arms CN, CP, P, CL, CL-W, CCP, P-D and B2 carry (the evaluability-floor row above): a cell with fewer than five valid outputs is not scoreable on its own; pooled counts are reported beside it
dead zone	clearance above zero and under the 20% bar: a count that is neither the registered zero nor a clearance at rate
template anchor $T(k^*, N)$	the truncated-grid value at the nearest-square order, $N/(2k^*)$, the anchor of the truncate branch (§2.3); at a trap cell it sits below the family argmax
grid order k	the k of a $k \times k$ grid, one circle per cell centre at $r = 1/(2k)$; k^* is the nearest-square order, and a packing's grid order is backed out of its dominant emitted radius (Supplement S9.2)

The clearance rule, stated once. Five thresholds appear: 10^{-6} and 10^{-9} for validity, 2×10^{-3} for value matching, 10^{-6} again as a minimum excess, and 5×10^{-8} as the granularity at which sums are recorded. One rule governs every claim that an output *left the family*:

An output leaves the family iff it is valid at 10^{-9} *and* its sum exceeds the family argmax by more than 10^{-6}.

Each condition excludes a real set of rows. The validity condition excludes eighteen rows of the conditioning arm (arm MU), a post hoc re-read. They sit above the argmax at the primary tolerance by at most 1.5×10^{-6} yet fail the tighter gate; their excess over the argmax is the overlap the primary tolerance admits. The excess condition excludes the two square-arm rows above, at 3.0×10^{-7} and 3.5×10^{-9} over the family best, which is the argmax at the recorded precision. Across the arms swept before the library arm (arm CL), exactly three rows leave the family. Two are rows at the extend (converge) cell $N = 20$ of the extend-cell arm, from a

post hoc sweep; one is a Sonnet sample at $N = 31$; all three are out-of-family constructions. The library arm's Sonnet tier adds twenty-three more, and a post hoc lenient reading of the weak tier's library programs four more that the registered parser does not count. The 2×10^{-3} value window never carries a clearance verdict; §3.3 records the one case where it was mistakenly allowed to.

3 Main result: who clears the family argmax

Twenty-one registered arms appear in this paper, each reported whichever way it went, and the falsifiers that fired are flagged in place: three fired, at the extend cell (F-M1), in the trace arm's tie reading (Supplement S6) and at the Sonnet library cell (F-CL1), the last in the paper's favour, since that falsifier named clearance at two or more cells. Every prediction whose verdict this section reports was registered before the arm's first scored run, with one exception. The Sonnet direct arm's registration named 5 of 20 samples at $N = 13$ and 21 as already seen, with $N = 31$ fully blind. The weak math-only replication and the Sonnet math-only arm were registered after the first weak math-only arm's results were known and before their own sampling; that is disclosure, not a second exception, since their predictions preceded their runs. Readings marked post hoc were not registered. Commits, amendments and outcomes are in Supplement S10. Table 2 lists the eleven arms that carry the channel table (Table 1), the optimizer-alone baseline among them and arm B2 beside it; all twenty-one are in Appendix A. The remaining arms establish the mechanism, the serving path or the mode prediction of Supplement S9. The trace arm (arm T), the rectangle arm (arm G) and the top-alias arm (arm O) carry no verdict the table needs; they are reported in the supplement (arm O in Supplement S3). Arms whose verdicts rest on non-discriminating cells, on samples under their floor, or on a null that does not separate from the template-shape null have a row in the appendix ledger (Table 4) and full text in the supplement. The reproducibility section states that criterion.

Table 2: The eleven arms that carry Table 1, with the optimizer-alone baseline and its extension to larger trap cells (arm B2), which has no cell in that table: section (a letter is an appendix; Sn is a section of the supplement), proposer, design, registered test, outcome. Design gives cells × samples per cell. Registered tests are coded P (prediction), F (falsifier), S (secondary prediction) and R (replication rule), each expanded in its arm's section. All twenty-one registered arms are in Appendix A (Table 4).

Arm	§	Proposer(s)	Design	Registered test	Outcome
F (square, bare)	3.3, S9	weak tier	7 N × 5–20	P1–P5 exact-value modes	modal at 7/7 N
S (Sonnet direct arm)	3.3	Sonnet tier	3 N × 10	P-S1–P-S4 tier contrasts	1/30 on-prediction, 100%/90% valid (10^{-6}/10^{-9}); one out-of-family escape; 5 of 20 samples at $N = 13/21$ seen before registration, disclosed
M (extend-cell arm)	3.3, S4	weak tier	4 N × 15	P-M1–P-M4 + F-M1/F-M2	F-M1 **triggered 3/3**: extend branch dies at $m \geq 4$; fifth k confirms; two out-of-family escapes at $N = 20$ (post hoc)
CC (weak math-only arm)	3.4	weak tier	3 N × 15	P-CC1 vs P-CC2	P-CC1 holds: anchor stays modal; 0/37 above the family argmax
CC2 (replication arm)	3.4	weak tier	3 N × 15	R1 + R2 + F-CC2.1	**REPLICATED**: 0/41 above argmax, anchor modal 3/3
CCS (Sonnet math-only arm)	3.4	Sonnet tier	3 N × 15	P-CCS1 vs P-CCS2	P-CCS2 holds: 0/37 above argmax — the channel, not the tier

Arm	§	Proposer(s)	Design	Registered test	Outcome
P (pinned path)	3.5, S8	weak tier, bare API	15 cells × 15	P-P1–P-P4 + F-P1	validity collapses: 0/105 valid at the square cells, 4/75 held-out; every direct-emission prediction UNSCOREABLE; P-P3 holds (0 of 12 valid programs clear); same-day agent-runtime control 6/6 valid, anchor 4/6 (post hoc)
CL (library code)	3.5	weak + Sonnet, numpy/scipy	2 tiers × 3 N × 15	P-CL1 vs P-CL2 + F-CL1	weak: 0/11 clear, P-CL1 holds; Sonnet: **23/25 clear at 3/3 cells**, P-CL2 holds, **F-CL1 fires**
CCP (isolating arm)	3.5	Sonnet tier, bare API, math only, 120 s	3 N × 15	P-CCP1 vs P-CCP2 + F-CCP1	P-CCP1 holds: 0/35 clear, none reaches the argmax; best per cell 1.625, 2.1, 2.673 — the library, not the path or the budget
CL-W (weak top-up)	3.5	weak tier, numpy/scipy	3 N × 15, pooled with CL	P-CLW1 vs P-CLW2 + F-CLW1; lenient reading secondary	P-CLW1 holds: 0/19 pooled clear, upper bound 16.8%; lenient reading 4/41, dead zone
B (optimizer alone)	3.5, S7	no model in the loop; a fixed SLSQP program	3 N × 15 seeds, 120 s	P-B1 vs P-B2 + S-B1	P-B1 holds: **45/45 clear**, 3/3 cells; S-B1: beats the Sonnet best at 3/3 cells; amendment 1 (version 1 blocked by the allowlist, kept)
B2 (optimizer alone, larger cells)	3.5 (program in S7)	no model in the loop; arm B's program, restart count changed	primary 2 N × 15 seeds, 120 s; secondary 3 N × 15 seeds, 3600 s	P-B2-1 + S-B2-1 + S-B2-2 + F-B2	P-B2-1 holds: **28/28 valid clear** of 30 launched at $N = 57$ and 59; S-B2-1 holds at 2/3 cells at 50 restarts; S-B2-2 not met at 2/3, $N = 91$ **UNSCOREABLE** (0 valid of 15 launched, all wall-killed); F-B2 not triggered

3.1 Registration protocol

For each cell the prompt is pinned and its SHA-256 digest written to disk first; the $N = 13$ square prompt hashes to `32db485b...`. Predictions live in the header of each arm's analysis script or in a standalone registration file. Competing predictions, numeric thresholds, tie conventions and a falsifier are fixed before the first invocation. Raw outputs are stored verbatim, failures included; scoring is deterministic and local. Supplement S9.2 records which of the bare square arm's seven cells carry an individually registered point prediction and which follow from the same closed form without one.

Four properties are disclosed, not repaired. Temperature, top-p and top-k are not exposed by the agent runtime. The alias-to-weights binding is a promise, not a hash. The subagent inherits instruction files outside the task prompt, so the digests lock a *fragment* of the conditioning context, and a replicator cannot reconstruct the inherited files from what we release. Hash coverage is incomplete, as the reproducibility section states. The pinned-path arm (arm P) and the library arm (arm CL) exist because of the first (decoding parameters not exposed) and third (inherited context outside the hash). They send registered prompts through a serving path that exposes and pins temperature and carries no inherited context.

Two scoring conventions were fixed in advance. Validity is reported at 10^{-9} and 10^{-6}, both logged, 10^{-6} primary. Proposers print six to eight decimals, so an eight-decimal tangency misses contact by $\sim 5 \times 10^{-9}$, while 10^{-6} sits far below the $\sim 10^{-2}$ gap between rival constructions. Value matching uses a 2×10^{-3} window, which never carries a clearance verdict. Every on-prediction rate is computed over *valid* outputs and so

conditions on execution success; the argmax-in-hand arm (arm CH) shows that this variable can separate what a model attempts from what it lands. Per-cell validity denominators are reported beside every rate so the reader can bound the exposure.

3.2 The table

Table 1 is where the tiers and channels are scored. Each row is one arm, or one registered pair of arms, and "clear" counts valid outputs under the clearance rule of §2.4. Six readings follow.

One row clears at rate. Sonnet-tier programs allowed `numpy` and `scipy` clear the family in 23 of 25 valid programs, at every cell. Their best program per cell and their median clearing program both beat the argmax, by the margins in the Sonnet library rows of Table 3. They sit above the grid family, not at the frontier: the same rows read below the published best-known sums at every cell. A fixed reference program with no model in the loop, run through the same pipeline, clears 45 of 45 and beats the best Sonnet program at every cell (arm B). The Sonnet row clears because its programs invoke that optimizer.

Neither tier without the optimizer; the optimizer without any tier. Weak-tier programs with the same libraries clear 0 of 19, pooled with their top-up (arms CL and CL-W), and the Wilson upper bound of that pool excludes the 20% bar. Two caveats, both carried by the weak library rows of Table 3. The $N = 21$ cell sits under the five-valid floor. A post hoc reparse that accepts `numpy` scalar output recovers 4 clearances of 41 valid, a point estimate whose upper bound does not exclude the bar. The weak tier clears at rate under neither reading, and the exclusion belongs to the registered reading alone. Sonnet-tier programs without the libraries clear 0 of 37 with simulated annealing over hand-rolled generators, Halton multi-restarts and force-directed relaxation, and 0 of 35 on the library row's own serving path and 120-second budget without reaching the argmax (arm CCP, the isolating arm); both are Sonnet math-only rows of Table 1. Every weak-tier row is 0 at the trap cells under the registered scoring. The optimizer-alone baseline that clears at every cell calls the same library the weak tier's valid programs call.

Escapes exist and are counted. Two cells show low-rate escapes, both out-of-family constructions found by sampling rather than by in-family search. One Sonnet direct emission in 30 clears at $N = 31$. Two weak-tier emissions in 15 clear at the extend cell $N = 20$, found in a post hoc sweep. They are in the table because a zero that hides them would be false. They do not change which cell clears at rate.

Rows are never pooled across serving paths. The same weak-tier prompt yields 35 of 45 valid packings, 78%, through the agent runtime and 0 of 105 through the pinned API (arm P). The clearing cell ran on the pinned path. The Sonnet math-only cell ran on the runtime, under a 10-second execution budget against the library arm's 120 seconds. Within the weak tier the library comparison is same-path: 0 of 12 math-only and 0 of 11 with libraries, both pinned, and 0 of 19 with the top-up. Each of those two arms has one cell above the five-valid floor, $N = 21$ for math-only and $N = 31$ for libraries, and both read 0 of 6 there. That comparison still crosses the budget, 10 seconds against 120. Within the Sonnet tier the isolating arm removes the crossing: the Sonnet tier, sent the math-only prompt the weak math-only arms used, on the pinned path under 120 seconds clears 0 of 35. The library's credit is that same-instrument gap, 0 of 35 against 23 of 25, on one path and one budget; the runtime figure of 0 of 37 crosses instruments and attributes nothing.

The same rows, per invocation. Every rate above conditions on validity, and validity is what the tier and channel manipulation moves, so Table 3 also charges every invocation, valid or not. That reading changes no registered verdict. The Sonnet tier clears 23 of 41 invocations, the weak tier 0 of 90, or 4 of 90 under the lenient reparse, and the baseline 45 of 45. The Sonnet denominator is 41 because the serving path refused four invocations with an HTTP 402, and a refusal is charged to the path. The lenient weak reading stays below the 20% bar on this count too, at a 95% Wilson upper bound of 10.9%.

Outside the table. Three arms sample regimes no row covers: five held-out cells absent from every scoreboard (arm CN), one parent-conditioning step (arm MU) and five generations of a proposer loop (arm L). None of their valid outputs clears the family argmax either; for arm MU that is a post hoc re-read, since no clearance prediction was registered for it. Pooled with the three math-only program rows (arms CC, CC2 and CCS), 0 of 290 valid outputs across six arms clear, the three code arms contributing 0 of 115 and the held-out arm 0 of 63, the arm-MU and arm-L rows entering from the companion paper's reports. That pool counts arm MU

at 10^{-9} and the other five arms at the primary tolerance. Held at 10^{-9} throughout it reads 0 of 281. Held at 10^{-6} throughout it reads 315 valid outputs with 18 above the argmax, every one of the 18 an arm-MU row whose excess is an overlap the tighter gate rejects. The pool, its membership rule and its tolerance accounting are in Supplement S1. The pool describes those arms and is not a bound: the escapes above sit outside it by named regime.

3.3 Direct emission

The weak tier emits a computable template. The bare square arm (arm F) produces uniform grids of radius $1/(2k)$ truncated to N circles, with interior-vertex fillers only when the grid underfills. Its modal output is the closed form's prediction at all seven tested N. At the four discriminating cells, $N = 13$, 21, 31 and 43, where prediction and family argmax differ, 41 of 57 valid emissions land on the prediction and 2 reach the rival argmax, both at $N = 21$. No valid emission at those cells clears the family. Supplement S9 holds the per-cell table, the template-shape null, the held-out replication at five N and the container and paraphrase probes.

Two weak-tier emissions do clear the family, the second count in the note of Table 1's first row. At the extend cell $N = 20$ the extend-cell arm (arm M), reported in Supplement S4, disconfirmed the registered prediction $V(4, 4)$: the model emits the 5×4 rectangle, $T(5, 20)$, in 12 of 15. Two of the fifteen valid rows are staggered rows of four at $r = 1/9$, sum 2.2222222, a hexagonal layout the family does not contain, valid at 10^{-9}. They are not the modal output, they were found post hoc, and they are counted.

The Sonnet tier perturbs and mixes. The Sonnet direct arm (arm S) was valid 30/30 at 10^{-6} and 27/30 at 10^{-9}, and almost never on-prediction; the on-prediction counts for the Sonnet and the weak rows at these cells, and for the weak rows once the trace study's bare rows are added, are in the ladder table of Supplement S3. Instead of truncating a uniform grid it perturbs one, with enlarged edge rows, hexagonal interior rows and two or three distinct radii in the same packing, 29/30 multi-radius, as Figure 1 shows. One sample at $N = 31$ emitted 27 circles at $r = 1/12$ with 4 corner circles at $r = 1/8$, summing to 2.7499999991, above the family argmax. Recomputed from its coordinates the construction is tangency-tight, with minimum pairwise and wall slack exactly zero, and it is among the samples valid at 10^{-9}. It is the direct-emission row's one clearance: 1 of 30 at the primary tolerance, 1 of 27 at 10^{-9}.

That sample also exposed a scoring defect. At $N = 31$ the family rival, 2.7485281, and this out-of-family value, 2.75, differ by 1.47×10^{-3}, inside the registered 2×10^{-3} window, so the value classifier counted the escape as a hit on the construction it exceeded. The window was chosen for the anchor comparison, where the nearest competing value is $\sim$0.15 away. Classified by structure instead, the Sonnet rival count at $N = 31$ is 2/10 and pooled 5/30. A third alias, addressed through a bare tier name we call `opus_alias`, is the top-alias arm (arm O), reported in Supplement S3 with the three-attractor ladder it completes.

3.4 Math-only programs: an executed program does not escape the family

No math-only program clears the family at either tier. The weak math-only arms (arms CC and CC2) put 0 of 78 valid programs above the family argmax and exactly two on it; the Sonnet math-only arm (arm CCS) puts 0 of 37 above it. The channel changes dispersion, not escape. Deployed loops elicit *programs*, and the code-free design had assumed a code channel would route construction to an executed optimizer; the first weak math-only arm measures that assumption. It ran the three trap cells $N = 13, 21, 31$ at $n = 15$ weak-tier invocations per cell. The prompt is the bare stem with exactly two registered line replacements: the code-free clause becomes a program contract, standard-library `math` only, and the output clause asks for program source. Programs run sandboxed under `python -I` with a 10-second timeout and a registered AST allowlist; the executor and its four-bin smoke test sit in the registration commit. Stdout is scored by the bare square arm's conventions unchanged. Two predictions competed: the modal valid executed output remains the $T(k^*, N)$ bucket at 2 or more of 3 cells, or it sits strictly above the anchor at 2 or more of 3. Ties or mixed modes count as neither. The table gives, per cell, the valid count of 15, the modal value bucket with its count and its margin in samples over the runner-up bucket, the anchor rate and the outputs above the anchor.

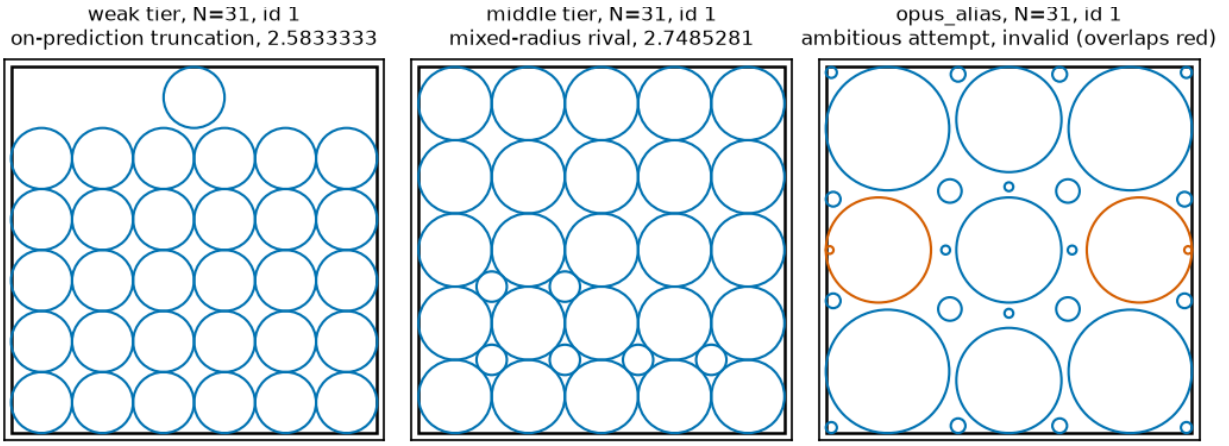

Figure 1: *Three attractor families at a single cell, $N = 31$.* One representative packing per tier, ledger sample id in each panel title (weak tier `bare` id 1, on-prediction truncation 2.5833333; Sonnet tier `sonnet_bare` id 1, mixed-radius rival 2.7485281; `opus_alias` id 1, invalid, with only the offending circles highlighted). Source: `arm_f_candidates_v2.jsonl` via `fig_scripts.py`, deterministic.

N	Valid	Modal bucket (count, margin)	Anchor-rate	Above anchor
13	12/15	anchor 1.6250000 (2/12, margin 1)	2/12	2 — 1.6614054, and the family argmax exactly
21	13/15	anchor 2.1000000 (9/13, margin 8)	9/13	0
31	12/15	anchor 2.5833333 (3/12, margin 1)	3/12	0

The anchor stays modal at 3 of 3 cells, on thin margins. At $N = 21$ the mode is heavy: most programs write the 5×5 grid truncated to 21 circles and print the anchor exactly. At $N = 13$ and $N = 31$ the modal bucket is the anchor value with a one-sample margin over dispersed sub-anchor values, so the modal-identity claim is thin at two of three cells; the table above carries every count and margin. The robust result is the registered above-argmax count: no valid program output exceeds the family argmax anywhere, 0 of 37 above the rival. Only two exceed the anchor at all, both at $N = 13$: one at 1.6614054, and one program that computes the family argmax construction $V(3, 4)$ *exactly*. Of the 45 programs, 7 produce geometrically invalid packings, 1 imports `random` against the stated contract and is caught by the registered gate, and none times out. Code-free anchor concentration at these cells runs 56 to 80%, the code channel's anchor rate is the anchor-rate column of the table above, and the lost mass sits entirely *below* the anchor: greedy radius-growth heuristics converging under the lattice.

Both registered follow-ups hold, and they harden the result. A fresh-draw replication wave (arm CC2) and the Sonnet math-only arm (arm CCS) were registered together, after the first wave's results were known, on the byte-identical prompts. The replication returned REPLICATED under its pre-stated verdict map: 0 of 41 valid outputs above the rival, exactly the registered zero-count prediction, and the anchor bucket modal at 3 of 3 cells. That includes $N = 31$, where the mode that was one-sample-thin in the first wave arrived at 10 of 15 valid with a margin of 8. The two pooled programs that reach the argmax come one per wave, both at $N = 13$. The Sonnet math-only arm's zero-count prediction held against a registered escape prediction, 20% or more of valid outputs strictly above the rival: no valid Sonnet-tier output exceeds the argmax. Its programs are qualitatively different search, simulated annealing over hand-rolled linear-congruential generators, Halton-sequence multi-restarts and force-directed relaxation, against the weak tier's greedy radius growth. The tier barely anchors, 5 of 37 on the anchor value, yet its executed math-only search never passes the family argmax at any cell. The replication returned 41 valid, 3 geometry-invalid and 1 blocked import; the Sonnet arm 37 valid, 7 geometry-invalid and 1 timeout. One replication invocation, $N = 13$ slot 2, used runtime tools despite the no-tools wrapper; it is scored as emitted, and the deviation table records it. Ledgers and scripts for these arms are listed in Appendix C.

3.5 Library programs: the channel with numpy and scipy

A Sonnet-tier program allowed `numpy` and `scipy` clears the family in 23 of 25 valid programs, at 3 of 3 cells. A fixed `scipy.optimize` program with no model in the loop clears 45 of 45 through the same pipeline. A Sonnet-tier math-only program on the same path and budget clears 0 of 35 and never reaches the argmax. A weak-tier program with the same libraries clears 0 of 19 under the registered parser, and its clearance rate is bounded under the 20% bar; a post hoc reparse reads 4 of 41 valid, 4 of 90 invocations. The clearing is the optimizer's: the Sonnet tier clears because its programs drive that optimizer, and a weak-tier program calling the same optimizer does not clear at rate. Table 3 holds every per-cell count, best sum and prediction verdict.

The arm. The library arm is the weak math-only arms' prompt with one clause changed: the import allowlist widens from `math` to `math`, `numpy` and `scipy`, which is the channel deployed loops permit. It ran at two tiers, the vendor's `claude-haiku-4.5` and `claude-sonnet-4.5` aliases, at temperature 1.0, on the pinned path of the pinned-path arm. Each tier sampled the three trap cells at $n = 15$ per cell, 90 invocations in all. The serving path refused 4 of them, all Sonnet-tier, with an HTTP 402, recorded as refusals; 85 returned a parseable program and one Sonnet response failed to parse. Each program runs in a fresh `python -I -S` subprocess under a 120-second wall clock, scored by the bare square arm's conventions, with clearance decided only by the clearance rule. Supplement S7 gives the fixed driver that runs the child and records the execution-mechanism correction. Each tier carried two competing predictions: no valid output clears, against clearance in at least 20% of valid outputs. The Sonnet tier also carried a falsifier, clearance at two or more of the three cells, with its integration rule written down in advance.

Weak tier: none clears, underpowered. The weak tier returned 11 valid programs of 45, every one calling `scipy.optimize`, and none clears the family. The 34 failures are 22 stdouts the registered parser could not read, 7 invalid packings, 4 execution errors and 1 blocked import; none timed out. The no-clearance prediction holds. Only $N = 31$ is above the five-valid floor, and it reads 0 of 6. The Wilson upper bounds, in the weak library row of Table 3, are both above the 20% bar the rival prediction set. The arm shows the weak tier did not clear, not that it cannot clear at rate; the top-up below powers the cell.

Post hoc: what the 22 unreadable outputs contain. All 22 parse failures print their packing as `numpy` 2.x scalar reprs, `np.float64(0.49...)`, which the registered parser's `literal_eval` rejects. No Sonnet-tier program did this. Stripping that wrapper, and nothing else, parses all 22 over a re-execution that reproduced every registered bin. Eleven of the 22 call `numpy.random` unseeded, so their sums belong to that execution. Of the 22, 9 are valid at both tolerances. The other 13 are invalid packings: 7 overlaps, 4 outside the square and 2 with non-positive radii. One clears, a deterministic $N = 13$ program with 13 distinct radii summing to 1.812531608, 2.0% above the argmax. Under this reading the weak tier is 1 of 20 pooled, the 20 being its 11 registered-valid programs plus the 9 recovered here, and 1 of 6 at $N = 13$, a rate in the dead zone. The registered count stands as the arm's result; the lenient count is reported beside it because a zero that depends on a print format is the wrong thing to lean on.

The weak-tier top-up (arm CL-W). The top-up was registered after the reparse, with the pooling rule, both readings and its predictions fixed in advance. It reruns the weak-tier prompt byte-identical, same path and decoding, for 45 more invocations, 15 per cell; the registration drafted 90, the credit allowed 45, and the deviation table records the cut. The 90 rows of both runs are scored in one pass by the library arm's pipeline. Of the 45 new rows 8 are valid. The failures are 23 unreadable stdouts, again `numpy` scalars, 4 invalid packings, 3 execution errors, 3 blocked imports, 3 timeouts and 1 with no program. Under the registered parser the pooled weak tier holds 6, 3 and 10 by cell, all valid at 10^{-9}, and 0 of 19 valid clear. The $N = 21$ cell stays under the five-valid floor and claims nothing on its own. The Wilson upper bound on the pooled count excludes the 20% bar, as the weak pooled row of Table 3 records. The no-clearance prediction holds and the falsifier does not fire. Under the post hoc lenient reading 41 are valid and 4 clear, a rate inside the dead zone; the lenient rows of the same table carry its bound. The last of the four lenient clearing sums, 2.851728045 at $N = 31$, is a sum the Sonnet tier also reached. An independent re-scoring with its own parser reproduced every bin and three of the four sums to the digit; the 1.784872559 program is unseeded, and its sum moves with the draw. The weak tier does not clear at rate under either reading.

Sonnet tier: 23 of 25 clear. The Sonnet tier returned 25 valid programs of 45 invocations (41 after the four HTTP 402 refusals) at the primary 10^{-6} tolerance, every one calling `scipy.optimize`. Of those 25, 23

Arm, reading	$N = 13$	$N = 21$	$N = 31$	Pooled; prediction verdicts
Clear of valid, by cell				
weak library (CL)	0 of 3	0 of 2	0 of 6	0 of 11; Wilson upper 25.9% pooled, 39.0% at $N = 31$; P-CL1 holds, both bounds above P-CL2's 20% bar
weak library (CL), lenient, post hoc	3 recovered, 1 clears	4 recovered	2 recovered	1 of 20; dead zone
weak top-up (CL-W), new rows	3 valid	1 valid	4 valid	8 valid of 45
weak pooled (CL + CL-W)	0 of 6	0 of 3	0 of 10	0 of 19; Wilson upper 16.8%, under the 20% bar; P-CLW1 holds, F-CLW1 does not fire
weak pooled, lenient, post hoc	2 clear	1 clears	1 clears	4 of 41, 9.8%, upper bound 22.5%; dead zone
Sonnet library (CL)	7 of 9	11 of 11	5 of 5	23 of 25; P-CL2 holds, F-CL1 fires at 3 of 3 cells
isolating arm (CCP)	0 of 10	0 of 14	0 of 11	0 of 35; P-CCP1 holds over P-CCP2, F-CCP1 does not fire
optimizer-alone baseline (B)	15 of 15	15 of 15	15 of 15	45 of 45; P-B1 holds over P-B2, S-B1 holds at 3 of 3 cells
Clear per invocation, every call charged				
weak pooled (CL + CL-W)	0 of 30	0 of 30	0 of 30	0 of 90; Wilson upper 4.1%
weak pooled, lenient, post hoc	2 of 30	1 of 30	1 of 30	4 of 90; Wilson upper 10.9%, under the 20% bar
Sonnet library (CL)	7 of 15	11 of 14	5 of 12	23 of 41; 4 HTTP 402 refusals charged to the path
optimizer-alone baseline (B)	15 of 15	15 of 15	15 of 15	45 of 45
Best sum per cell				
family argmax, closed form	1.776142375	2.258883476	2.748528137	the bar a clearing sum exceeds by more than 10^{-6}
published best-known, lower bound	1.829	2.362	2.889	three truncated decimals, from the bound table
Sonnet library (CL)	1.820699211	2.340549845	2.864990189	2.5%, 3.6% and 4.2% above the argmax; 0.5%, 0.9% and 0.8% below best-known
isolating arm (CCP)	1.625	2.1	2.672523443	all under the argmax; at $N = 13$ and 21 the anchor $T(k^*, N)$
optimizer-alone baseline (B)	1.829542412	2.359585843	2.883035274	above the Sonnet best at 3 of 3 cells; 1.8295 matches 1.829+ at $N = 13$; 0.1% and 0.2% below best-known at $N = 21$ and 31
weak pooled, lenient clearing sums	1.784872559, 1.812531608	2.331990574	2.851728045	post hoc; the 1.784872559 program is unseeded

Table 3: **Library-channel arms, cell by cell.** Every count, best sum and prediction verdict of this subsection. "Clear" counts valid outputs above the family argmax under the clearance rule. Wilson bounds are 95% upper bounds on the clearance rate. Lenient rows re-read the same outputs with a parser that accepts `numpy` scalars. For arm CL that reparse is post hoc and carries no registered verdict; for the top-up (CL-W) the lenient reading was fixed in its registration as the secondary reading, which the pooled lenient row reports. The per-invocation block charges every call, valid or not, and carries no registered verdict of its own: the registered readings are the clearance rates over valid outputs above it. Arm B2 carries the same baseline to larger cells, which have no column here. Budget-matched at the same 120-second wall: 14 of 15 launched rows valid and 14 of 14 valid rows clear at $N = 57$, one wall-killed; 14 of 15 launched valid and 14 of 14 valid clear at $N = 59$, one geometrically invalid. Restart-matched at 50 restarts under a 3600-second wall: 15 of 15 launched valid and 15 of 15 valid clear at $N = 57$; 11 of 15 launched valid and 11 of 11 valid clear at $N = 73$, four wall-killed; and at $N = 91$ 0 clear of 0 valid and of 15 launched, all 15 wall-killed, the cell UNSCOREABLE.

clear the family, and each of the 23 is valid at 10^{-9} as the clearance rule requires. The two non-clearing programs, both at $N = 13$, are also valid at 10^{-9}; their sums are 1.5 and 1.767686476, under the argmax. The count is therefore the same at either tolerance. The best program per cell exceeds the argmax; the per-cell excesses, their range and their median are in the Sonnet library rows of Table 3 and Table 1. The 20% prediction holds and the falsifier fires at every cell. Against the published best-known values, three-decimal

lower bounds from the bound table of Supplement S9.1, the best programs sit below at every cell, by the margins the same rows carry.

The isolating arm (arm CCP). Between the Sonnet math-only cell and the library cell, three things changed along with the import allowlist: the serving path, the dispatch context (the runtime's wrapper and inherited instruction files) and the execution budget. The math-only arm CCS ran on the agent runtime under a 10-second budget; the library arm ran on the pinned API under 120 seconds. Neither cell alone said which factor cleared. The isolating arm reruns the weak math-only arms' prompt byte-identical on the library arm's path: Sonnet tier, pinned API, temperature 1.0, no system prompt, a 120-second budget, `numpy` and `scipy` blocked by the AST gate. Its competing predictions were no clearance (the library is what clears) against clearance at 20% or more at two of three cells (path or budget explains the library arm). Of 45 invocations, one was refused mid-run at row 24; the run resumed with a single worker process, the request unchanged, and the refused row was re-issued. Valid at both tolerances are 35 programs, 10, 14 and 11 by cell, and 0 clear; 10 are geometrically invalid, with no timeouts and no blocked imports. None reaches the argmax, and at $N = 13$ and $N = 21$ the best program is the template anchor $T(k^*, N)$ itself. The anchor's structure is modal: 7 of 10 programs at $k = 4$ and 12 of 14 at $k = 5$. The no-clearance prediction holds and the falsifier does not fire. The three Sonnet program cells now read 0 of 37, 0 of 35 and 23 of 25, and only the third had the libraries. Within the Sonnet tier, path and budget moved nothing and the libraries moved everything. At the weak tier the same libraries move nothing at rate. Leaving the family at rate took the tier *and* the optimizer.

The optimizer alone (arm B). The control the title asks for is a fixed `scipy.optimize` program with no model. The optimizer-alone baseline was registered before its first scored run; Supplement S10 records its post-review motivation. Its competing predictions: the program clears at 20% or more of valid runs at two or more cells, or it never clears. Under the first, the optimizer alone clears and the introduction's first contribution is rewritten by a rule fixed in the registration: the contribution, until then "the clearing took a mid-tier model and a library optimizer", becomes "the library optimizer clears the family; the tiers differ in whether their programs drive it there", the abstract's headline sentence follows, and the Sonnet programs are characterised against the baseline. Under the second the conjunction stands. A secondary prediction asked whether its best packing per cell exceeds the Sonnet best. The program is random-restart SLSQP over centre and radius variables with analytic gradients, uniform random starts, a uniform radius shrink by the largest constraint violation, and 50 restarts. Its 94 lines, and the two versions the registration went through, are in Supplement S7; Claude drafted it under the human author, and no result informed either version. "No model" means no model call at proposal time: one fixed program serves every N and seed, where every other program cell in the channel table is written afresh by a model at each invocation. It ran unmodified through the library arm's pipeline and scoring, on one core, at the three cells with 15 seeds each. Result: 45 of 45 valid at both tolerances and 45 clear, 15 of 15 at every cell. Its best sum per cell beats the Sonnet best at 3 of 3 cells, so the secondary prediction holds. At $N = 13$ its 1.8295 matches the published lower bound, 1.829 to the three digits published. Its packings are off-family: dominant grid orders 3 to 7 across cells, no single template. The first prediction holds and the integration rule applies: the library optimizer clears the family with no model in the loop.

The optimizer alone at larger trap cells (arm B2). An external review of revision 5.16 asked whether "the optimizer does the clearing" survives where restart SLSQP no longer saturates the cell. Arm B2 answers it: registered before any row was sampled (`00a1b69`), with no model in the loop, it re-runs arm B's program byte-identical except for its restart count, seeds 1–15 per cell, on a 22-core machine that is not arm B's. One result needs no scored run: arm B's 45 of 45 used 50 restarts inside a 120-second wall at $N = 13$, 21 and 31, while the same frozen program in the same wall gets 7 restarts at $N = 57$, 5 at $N = 59$, 1 at $N = 61$, and cannot finish one restart at $N = 73$, where seed 3 needs about 223 seconds for one restart, reproduced to the second. Contribution 1's claim is therefore budget-scoped, whatever the scored rows say. The primary reading is budget-matched, arm CL's pipeline unmodified at that same wall with the restarts that fit: at $N = 57$, 14 of 15 launched rows are valid, one wall-killed, and 14 of 14 valid rows clear, best 3.893099568 over the family argmax 3.736693464 and the anchor $T(8, 57) = 3.5625$; at $N = 59$, 14 of 15 launched rows are valid, one geometrically invalid, and 14 of 14 valid rows clear, best 3.968111658 over 3.79586683 and 3.6875. Pooled, 28 of 30 launched rows are valid and 28 of 28 valid rows clear: P-B2-1 holds, and the best

exceeds the anchor at both cells. The secondary reading is restart-matched, 50 restarts under a 3600-second wall, off-pipeline by design and carrying no claim that its rows are scored as the model-written programs are: at $N = 57$, 15 of 15 launched rows are valid and 15 clear, best 3.917833616; at $N = 73$, 11 of 15 launched rows are valid, four wall-killed, and 11 of 11 valid rows clear, best 4.43000727 over 4.232995129 and $T(9, 73) = 4.0555556$; at $N = 91$, 0 clear of 0 valid and of 15 launched, all 15 wall-killed at 3600 seconds, so that cell is UNSCOREABLE under the five-valid floor, with its argmax 4.730118646 and anchor $T(10, 91) = 4.55$ untested. S-B2-1 holds, clearance at 2 of 3 cells at 50 restarts; S-B2-2 is not met at 2 of 3, cell 91 having no valid row to set against its anchor; F-B2 is not triggered. Cell 91 is a reading about budget, not about reach; Supplement S7 gives the registration's timing for that cell and the concurrency the run used. Per the registration, a gap between the two readings is a statement about restart budget, not about the optimizer's reach: cell 91 is not evidence that the optimizer fails there. At $N = 91$ the family argmax rests on the closed form alone, since the LP oracle covers $k \leq 9$ and the exhaustive sweep stops at $N = 80$; nothing turns on it, because the cell scored no row. Supplement S10 records the worker count the secondary ran at.

What the arms establish. Within the Sonnet tier the library is the factor, and with the optimizer-alone baseline the library is sufficient. A weak-tier program invokes the same library in every one of its 19 valid programs and clears nothing at rate, and no math-only program clears at either tier, on either path or at either budget. The tiers differ in whether their programs drive the optimizer to a clearing packing, the execution finding the mechanism section reports for the code channel. The 10-second budget of arms CC, CC2 and CCS remains a difference from the library arm for the weak tier's own contrast. Per the registered integration rules, the abstract and title name the optimizer as what clears, and the library-enabled Sonnet tier as the only model-written regime that drives it there. The isolating arm's 35 valid programs are reported by name, not pooled with the 0 of 37. Ledgers and scripts are listed in Appendix C.

The pinned-path arm (arm P): the registered prompt on a bare serving path. This arm ran $n = 15$ per cell at the seven square, five held-out and three code cells, with four predictions and one falsifier; Supplement S8 gives the serving path it pins and the registration in full. Validity collapsed: 0 of 105 outputs at the square cells are valid at either tolerance, and 4 of 75 at the held-out cells, none clearing. The direct-emission predictions are unscoreable under the arm's floor and the falsifier's denominator is empty. On the code cells 12 of 45 programs are valid and none clears, so the code-channel prediction holds. Two post hoc diagnostics: at $N = 13$, temperatures 0.0, 0.5 and 1.0 each give 0 of 6 valid. The same prompt through the agent runtime, same day, gives 6 of 6 valid and lands the anchor $T(4, 13) = 1.625$ in 4 of 6.

The thinking-budget arm (arm P-D) names the component. It sends the same $N = 13$ prompt on the pinned path, $n = 15$ per condition: D1 turns extended thinking on; D2 adds one system line asking for care. The prediction: D1 valid in at least 8 of 15, D2 in at most 3. D1 is valid in 12 of 14 and D2 in 0 of 13 (the run closed early by amendment, Supplement S10), so it holds, and the anchor $T(4, 13) = 1.625$ is modal among D1's valid outputs at 5 of 12. The anchoring result is a property of its instrument, and the component is extended thinking, a request parameter on the pinned path.

4 Mechanism: choice follows the score table, execution does not

Table 1 says a weak-tier proposer leaves the family at rate in no channel, and a Sonnet-tier one leaves it at rate only in the library cell. Two readings of that are open: the weak tier *prefers* the template, or it *cannot build* anything else. One registered arm separates them, and three other directions show the same split. The predictions whose verdicts this section reports were registered before their arm's first scored run; the readings marked post hoc were not. Their outcomes are in Appendix A, and every post-hoc reading below has a row in the deviation table.

4.1 Arm CH: the argmax in hand does not dislodge the template

Handed the argmax, the model still emits the template, yet reaches for the argmax. Under the registered scorer the template prediction held at 2 of 3 cells. Counted at the attempt level, post hoc, the model reaches for the argmax in 36 of 45 invocations and builds it in 6 of 14 valid outputs.

The argmax-in-hand arm (CH) hands the model the bare prompt plus every admissible $T(k, N)$ and $V(k, m)$ value, the family argmax included, and tells it to use whichever scores highest or anything better. It ran 15 invocations per cell at $N = 13$, 21 and 31, 45 in all, registered together with the conditioning arm (MU) before sampling. The tractability alternative of the limitations section says the model truncates at k^* because it can compute nothing else. Enumerating the family removes that burden, so under it the modal output should become the stated argmax; under the template reading it stays $T(k^*, N)$.

The registered verdict carries a survivorship artifact, reported rather than exploited. The registered scorer takes the modal output among *valid* samples, and under it the template reading wins. The stated argmax, the modal valid output and the verdict at each cell are the table below. The modal output stays $T(k^*, N)$ at $N = 13$ and at $N = 31$, the second by a one-sample margin, and only $N = 21$ goes to the argmax, so the argmax prediction held at 1 of 3 cells. This arm set no floor of its own and grants $N = 13$ a verdict on two valid samples. Under the GM chain's floor of three that cell would not be scoreable, and the template prediction would hold at 1 of 2. Both readings are reported, since each floor was fixed at its own arm's registration (§2.4). But validity is low, as the table's valid column shows, and the failure modes change the picture at the *attempt* level.

The attempt-level reading, post hoc. Of the 31 invalid rows, 30 are evaluable. **All 30 attempt the argmax at its own grid order** k^*-1: 12 of 13 invalid rows at $N = 13$, 10 of 10 at $N = 21$ and 8 of 8 at $N = 31$. The order is backed out of each row's dominant emitted radius, as Supplement S9.2 recovers k. The remaining row emits radical literals the scorer cannot evaluate and is not scored as attempting anything. All of them misplace the fillers, typically at the container corners, where radius $(\sqrt{2} - 1)/(2k)$ overlaps the adjacent grid circle instead of sitting in a grid interstice.

The control this reading needs. Attempt counts need the unprompted rate. The bare square arm (F) supplies it: its invocations at the same three cells, under the same dominant-radius estimator. Over *all* evaluable rows, valid and invalid alike, the bare square arm reaches for the argmax order in **5 of 57** against the argmax-in-hand arm's **36 of 44**. Its one-sided Fisher p, post hoc and uncorrected, is in Supplement S10 (`arm_f_attempt_uncond.py`). Those 57 are the bare square arm's evaluable rows at $N = 13$, 21 and 31, not the 57 valid emissions of the direct-emission subsection's four cells. The invalid-row comparison conditions on a variable the manipulation itself moves, a collider: the argmax-in-hand arm is 31 of 45 invalid and the bare square arm 10 of 60 at the same cells. For the record it points the same way; its counts and its p are in the same table (`arm_f_attempt_control.py`). Both comparisons are post-hoc strata, not registered outcomes.

What the arm supports. The 36 of 45 attempts are the 30 evaluable invalid attempts plus 6 valid outputs that land the argmax; the control's denominator drops the one unevaluable row, as the arm CH row of Supplement S10 records. The model builds the argmax in 0 of 2 valid at $N = 13$, 3 of 5 at $N = 21$ and 3 of 7 at $N = 31$, 6 of 14 pooled. It does not *fail* to build the argmax; it builds it unreliably. The registered metric conditions on execution success, the very variable that separates the two readings, so the summary is a decomposition rather than a verdict. Handed the argmax, the model's *choice* follows the score table. Its *execution* succeeds less than half the time it is attempted, failing off-template in 30 of the 31 invalid attempts on the post-hoc reading. The tractability alternative (the mechanism-free reading of §6) is wrong about selection and right about instantiation.

N	Stated argmax	Modal valid output	Valid	Invalid rows attempting argmax	Registered verdict
13	1.7761424	$T(4, 13) = 1.6250000$	2/15	all	P-CH1 holds
21	2.2588835	argmax, 3/5	5/15	all	P-CH2 (argmax)
31	2.7485281	$T(6, 31) \approx 2.5833$ (4/7)	7/15	all	P-CH1 holds (one-sample margin)
pooled	n/a	template 2 of 3 cells	14/45	**30/30** evaluable (all misplace fillers; 1 further row unevaluable, radicals)	choice follows score table (36/45 attempt it); execution lands it 6/14 valid

Valid is valid of 15 invocations per cell. P-CH1 is the template prediction, the modal valid output stays $T(k^*, N)$; P-CH2 the argmax prediction, the modal valid output becomes the stated argmax.

4.2 The same asymmetry from other directions

In code. An executed math-only program is the cleanest removal of the hand-computation burden. It recovers the argmax construction exactly twice in 78 valid weak-tier programs, one per wave, both at $N = 13$ and each computing $V(3, 4) = 1.7761424$, and it never passes it (the math-only subsection). With `scipy.optimize` available the weak tier clears nothing in 19 valid programs under the registered parser, and 4 of 41 under a post-hoc lenient one. The Sonnet tier clears 23 of 25, and 0 of 35 without the libraries on the same path and budget. Every valid weak-tier library program calls `scipy.optimize`, and none drives it past the family. The library supplies execution the weak tier invokes but does not steer to a clearing packing; the optimizer-alone baseline (B), a fixed program with no model, steers it there at 45 of 45.

At a held-out cell. The held-out-cells arm (CN) has one miss, $N = 50$, and it is the argmax-in-hand pattern in the wild. There 10 of 11 valid outputs are the registered 7×7 grid plus exactly one filler, and 6 of them push the filler into a container corner instead of the interstice. The rows are Supplement S9.4. The template is selected; the filler is mis-executed; the value criterion registers a miss.

Across a vendor, from the companion paper. A second weak-tier family, gemma-4-26b in the gemma arm (GM3) on a serving path that pins temperature, emits the rival argmax itself as its modal output at every discriminating cell. A post-hoc structural diagnostic finds every rival-valued discriminating-cell packing carrying the rival's exact two-radius decomposition, 27 of its 43 valid outputs there against the original family's 2 of 57 at its own, those 57 being the valid emissions of the direct-emission subsection's four discriminating cells; the cells are in the companion paper, whose evidence this direction cites and does not re-score. A family with more arithmetic reach executes the harder in-family construction instead of truncating: same recipe family, other branch.

Not counted as a direction: the top alias. A third nominal tier attempts a more ambitious construction at every cell than either other tier and is valid in 4 of 30 invocations. Its alias-to-weights binding is not attestable and four valid samples cannot carry a tier statement, so it supports nothing here. Supplement S3 reports the arm, the ladder table and the failure taxonomy in full.

The three directions share the order of failure. Selection is not the bottleneck: the weak tier picks the argmax when shown it, and a family with more reach emits it unprompted. Execution is: the picked construction is mis-built in text, reached but not passed in math-only code, and cleared at rate only where a Sonnet-tier model hands the arithmetic to an optimizer. The template is what survives execution.

5 Related work

LLM-driven discovery and the circle-packing scoreboard. Language Model Crossover (Meyerson et al., 2024), ELM (Lehman et al., 2022), EvoPrompt (Guo et al., 2024), EoH (Liu et al., 2024) and LLaMEA (van Stein & Bäck, 2025) established the LLM call as an EC operator over prompts and programs. FunSearch turned the pattern into a discovery claim, and every system in the introduction's scoreboard inherits it. HELIX's figure is marginally *below* ShinkaEvolve's relaxed-tolerance figure and above its strict one, yet is described as state of the art. ThetaEvolve claims new best-known bounds with a printed $N = 26$ figure equal to HELIX's. The benchmark is saturated across the AlphaEvolve–ShinkaEvolve–HELIX cluster, and the claims outrun the digits. OpenEvolve (Sharma, 2025), the open-source AlphaEvolve reproduction most practitioners run, reports 2.634292 in its published circle-packing example at the time of access; the repository is unpinned. Supplement S9.1 records the correction to our record comparison and its $N > 40$ coverage gap. Two critiques bear on our framing and are often conflated, so we attribute them separately: Gideoni et al. (2026) show simple baselines recover much of the reported advantage, and Berthold et al. (2026) show classical solvers do too.

Template convergence and diversity collapse. "Mutation Without Variation" (Gurkan et al., 2026) finds iterated LLM program mutation collapsing onto previously seen structural templates: in 87% of chains, over 93% of mutations revisit a seen form. Same bias family, different regime: mutation loops, program space, no closed form, no preregistration. It is the closest published neighbour to the companion paper's loop arm (L). That arm's second registered prediction, reported there, confirmed the analogous statement inside a

scored loop: iterating the archive moves the anchor, and no valid output clears the family argmax. Their result is about structural revisiting; ours is about a value bound the revisited structures cannot cross. On anchoring, Huang et al. (2026) treat it as a general bias over initial information, Lou & Sun (2024) document numeric primes dragging point estimates, and Malberg et al. (2025) test it on a numeric prime among thirty biases. Our modality differs: a construction template has a computable value where a scalar anchor has only a measurable pull. "Measuring the Gap Between Human and LLM Research Ideas" (Chen et al., 2026) reports the same collapse one level up, in idea space. Against 11,683 human ideas, LLM ideas concentrate on bridge-type opportunities, 47–64% versus 12% for humans, with normalized entropy down to 0.55 versus over 0.92. "Artificial Hivemind" (Jiang et al., 2025) documents the same homogeneity across open-ended domains at survey scale. Those results are distributional; ours is the limiting case in which the collapsed mode is a single closed-form-computable object, stated before sampling. Converging skepticism about what the proposer contributes comes from "Dictionaries, Not Darwin" (Li, 2026), Wang et al. (2026), BehaveSim (Zhang & Lu, 2026), Strategy Diversity (Yang et al., 2026) and the bin-packing critiques (Herrmann & Pallez, 2026; Sim et al., 2025). Two results cut the other way. Zhang et al. (2024) ground the *importance* of evolutionary search empirically, evidence against our substitution of an unconditioned call for the loop. Zhang et al. (2026b) finds strong LLM optimizers act as *local refiners*, which a parent-conditioned call would be. Both are engaged in the limitations section.

Scaling inversions. Zhou et al. (2024) show larger and more instructable models becoming less reliable. The o3/o4-mini system card (OpenAI, 2025) reports a higher PersonQA hallucination rate for o3 than for o1 alongside higher accuracy, attributed to more claims made. Shojaee et al. (2025) reports collapse past a complexity threshold. GeoBuildBench (Kim et al., 2026) finds geometric construction a regime where nominal capability and executed correctness diverge, and MathConstruct (Balunović et al., 2025) benchmarks constructive proofs, where the object built is checkable. Our instantiation is *attractor family* versus executability.

Preregistration lineage. Thomas et al. (2026), Williams et al. (2026), Vaccaro (2026) and Zhang (2026) preregister recipes, outcome-blinded predictions and agent protocols. HindsightBench (Jia, 2026) releases frozen preregistrations of directional aggregate hypotheses. We adopt the standard rather than claim it; what we add is the object locked, exact closed-form point predictions of specific output values. Structural precedents: Kaplan et al. (2020) and Hoffmann et al. (2022) publish a functional form then run the confirming instance; Snell et al. (2024) is the closest structural twin.

6 Limitations and untested alternatives

What the companion paper measures. Six registered arms changed the setting in ways this paper does not report: the conditioning arm (MU), the loop arm (L), the cross-vendor arm (V) and the three second-vendor arms (GM, GM2 and GM3). The template is contingent on the serving path and the request parameters it carries, the reasoning setting included; it is neither confined to one instrument nor invariant across them. A companion paper reports those arms in full and measures where the characterization holds and where it breaks.

Scale. The related work's evidence that evolutionary search contributes (Zhang et al., 2024) and that strong LLM optimizers act as local refiners (Zhang et al., 2026b) is answered by two arms of the companion paper. The conditioning arm (MU) takes one conditioning step, and the loop arm (L) runs five generations of a registered archive-and-mutate loop. What stays open is scale: population five over five generations at one tier, the companion paper's loop, is a loop, not the loops of the related work. The mechanism-free reading (the tractability alternative of §4.1), that the $k \times k$ grid is what survives mental arithmetic, was probed by the argmax-in-hand arm (CH) and split in two. Tractability does not explain the selection; on that arm's post-hoc attempt-level reading it does explain the instantiation.

Contamination is probed at one level only. $N = 26$ is the canonical published cell and plausibly in training data. The held-out-cells, container and recall arms (CN, CP and RP) bound the recall mechanisms; their rows are Supplement S9.4 and Supplement S9.5. They test held-out N, a container that is not the benchmark's, and a direct request for the published values. A planted-canary test is inapplicable to a

benchmark we did not author. Absence of the fitted cells from the pretraining corpus is established by none of it.

Vendor scope. All tiers in the main study come from one provider; the companion paper's cross-vendor arms move the pooled-level claim across vendors. One finding is itself a limitation: a cross-vendor null under a fixed output budget is not evidence about the model until the budget's interaction with the vendor's response structure is checked. The 4k-budget gemma arm (GM2, one of the six arms the companion paper reports) returned, per the companion paper's report, 0 of 140 parseable at a 4,096-token budget. The same model at 16,384 tokens parses at 123 of 140, because the vendor API returns deliberation on a separate channel that the smaller budget died inside. After those arms the anchoring law is cross-vendor at the pooled level and does not separate from Supplement S9.2's template-shape null at the discriminating cells. Its full cell-level mode structure is confirmed only within the original family.

Two instruments, and the contrast that crossed them. The agent runtime does not expose decoding parameters, so every runtime row is a distributional claim over an observed sampling regime. The pinned rows bound that: the pinned-path arm (P), the library arm (CL), the isolating arm (CCP) and the weak top-up (CL-W). The pinned-path arm shows the two paths are not interchangeable for the weak tier, so Table 1 never pools across instruments. The Sonnet contrast crossed them: the Sonnet math-only arm (CCS), 0 of 37, ran on the runtime at 10 seconds, and the library arm, 23 of 25, on the pinned path at 120 seconds. The weak math-only arms (CC, CC2) share the 10-second budget. The 12× gap was registered on purpose, since a 10-second gate measures the timeout rather than the search; the Sonnet tier's programs can nonetheless hit that gate. The isolating arm removes the cross: the Sonnet tier, sent the weak math-only arms' prompt on the pinned path at 120 seconds, returns 0 of 35 clear, none at the argmax. The Sonnet gap is therefore attributed to the library alone. Within the weak tier the contrast is same-instrument, 0 of 12 and 0 of 11, both pinned, but crosses the budget: the pinned-path arm's code cells ran at 10 seconds. No weak-tier program in any arm has passed the argmax under the registered parser at either budget; the lenient reading of the library arms finds 4 of 41.

`opus_alias` provenance. The label denotes the serving alias addressed, not an identified model version. Without pinned weights we cannot separate a model-tier effect from a serving-path effect, and the supporting anomalies live only in the runtime transcript.

Prompt robustness. Every arm other than the paraphrase arm (PP) conditions on a single hash-locked prompt; locking buys reproducibility, not robustness to rewording. The paraphrase arm samples three semantic-preserving paraphrases at two discriminating cells, $N = 13$ and $N = 31$, and the registered prediction stays modal at all six paraphrase-cells, Supplement S9.5. So the anchor is not keyed to the literal template string at those cells.

Untested alternatives.

- Typicality bias in preference-tuned models (Zhang et al., 2026a) predicts concentration on the most typical construction without reference to geometry; no observation here excludes it.
- Membership and extraction probes of the kind used to detect benchmark contamination (Carlini et al., 2021; Golchin & Surdeanu, 2024) were not run.
- A formatspread-style sweep on the scale of Sclar et al. (2024) remains untested.
- Alignment-induced diversity loss (Kirk et al., 2024) is untested as the account of the cross-vendor concentration differences, which are measured but not explained.

7 Reproducibility, deviations, and disclosures

Provenance in brief; Appendix C carries it in full.

- **Registration and provenance.** Every arm's prompts were hashed and its predictions and falsifiers fixed before its sampling, with the exceptions the deviation table records. No arm was selected on its

result, and every arm is reported. The provenance appendix lists each registration with its commit, ledgers and scripts, the limits on what the prompt digests lock, a metadata correction to the shipped ledger, the excluded records, and the internal adversarial check. Deviations are tabled in Appendix B and stated in full in Supplement S10. The post-hoc tests in the body carry no multiplicity correction; their p-values are descriptive, not confirmatory.

- **Use of AI systems.** Claude models were used to draft manuscript prose and to assist implementation. The human author designed the study, set the research questions, specified the instruments and their decision rules, and approved each preregistration before sampling. The author made the final inclusion, stopping and submission decisions and is solely responsible for the content. The claim a reader can check is the ordering, not the authorship: each preregistration commit is a git ancestor of the sampling it governs. The drafting models share a family with arms under study, the Sonnet direct arm among them, so what a reader is asked to trust is the released artifact, not the author.

- **Artifact availability.** The complete artifact accompanies this submission as anonymized supplementary material: ledgers, analysis and reproduction scripts, LP oracle, every preregistration and amendment, frozen reports and the corrections ledger, with the supplement PDF and an anonymized copy of the companion paper (`companion_anonymized.pdf`). Every number regenerates via `HOW_TO_RUN.md` from the shipped ledgers. Only the pinned-path arms, P, CL, CCP, CL-W and P-D, can be re-sampled. The runtime rows depend on a dispatch wrapper and inherited context the digests do not lock, as the registration protocol states, so a reader can re-score them but cannot re-draw them. The working repository is a public remote, unlinked here for anonymity.

- **Which arms are in the body.** Every arm reported here has a row in Table 4 and a ledger named in the section that reports it or in the provenance appendix; the six companion-paper arms share one pointer row there. An arm has a body subsection when its registered verdict bears on the channel table, the mechanism or the serving path. Arms whose verdicts rest on non-discriminating cells, on fewer valid samples than their floor, or on a null that does not separate from a template-shape null are reported outside the body. The trace and rectangle arms (T and G) are supplement-only, in Supplement S6 and Supplement S5; the optimizer-alone baseline (B) has its reference program listed verbatim in Supplement S7. The cross-vendor and gemma arms (V and GM3) are reported in the companion paper, with the conditioning, loop and remaining second-vendor arms (MU, L, GM and GM2, listed in Supplement S2); this paper cites their findings where the mechanism section uses them. The extend-cell arm (M) keeps its registered verdicts and its per-cell decomposition in Supplement S4. The `opus_alias` post-hoc dual accounting is Supplement S3. The pooled-zero table of Supplement S1 pools four arms of this paper with two of the companion paper's and is not an arm. Applying that body-inclusion criterion changed no number in Table 1.

- **Stopping rule.** The manuscript closed after implementing the internal-check findings; no further data-dependent analysis was performed. The pinned-path arm (P) and the library arm (CL) were registered together after an external review and pushed before either was sampled; each was sampled and scored once. The pinned-path arm's two post-hoc diagnostics are labelled as such in Table 5. The arms registered after that close, each run and scored once under its own preregistration, are listed in order in Supplement S10. The analyses named but not run are recorded in Supplement S6.5 and Supplement S9.3.

References

Mislav Balunović, Jasper Dekoninck, Nikola Jovanović, Ivo Petrov, and Martin Vechev. Mathconstruct: Challenging llm reasoning with constructive proofs, 2025. URL `https://arxiv.org/abs/2502.10197`.

Timo Berthold, Dominik Kamp, Gioni Mexi, Sebastian Pokutta, and Imre Polik. Out-of-the-box global optimization for packing problems: New models and improved solutions, 2026. URL `https://arxiv.org/abs/2605.04850`.

Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *30th USENIX Security Symposium*, 2021.

Mert Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei Zaharia, Alex Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization, 2026. URL https://arxiv.org/abs/2602.20133.

Ziyu Chen, Yilun Zhao, and Arman Cohan. Measuring the gap between human and llm research ideas, 2026. URL https://arxiv.org/abs/2607.01233.

Yonatan Gideoni, Sebastian Risi, and Yarin Gal. Simple baselines are competitive with code evolution, 2026. URL https://arxiv.org/abs/2602.16805.

Shahriar Golchin and Mihai Surdeanu. Time travel in LLMs: Tracing data contamination in large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.08493.

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.08532.

Can Gurkan, Forrest Stonedahl, and Uri Wilensky. Mutation without variation: Convergence dynamics in llm-driven program evolution, 2026. URL https://arxiv.org/abs/2606.05408.

Julien Herrmann and Guillaume Pallez. An in-depth study of llm contributions to the bin packing problem, 2026. URL https://arxiv.org/abs/2510.27353.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022. URL https://arxiv.org/abs/2203.15556.

Yiming Huang, Biquan Bie, Zuqiu Na, Weilin Ruan, Songxin Lei, Yutao Yue, and Xinlei He. Understanding the anchoring effect of llm with synthetic data: Existence, mechanism, and potential mitigations, 2026. URL https://arxiv.org/abs/2505.15392.

Haozhe Jia. Hindsightbench: A black-box behavioral audit protocol for parametric hindsight in time-indexed llm decision tasks, 2026. URL https://arxiv.org/abs/2607.18867.

Liwei Jiang, Yuanjun Chai, Margaret Li, Mickel Liu, Raymond Fok, Nouha Dziri, Yulia Tsvetkov, Maarten Sap, Alon Albalak, and Yejin Choi. Artificial hivemind: The open-ended homogeneity of language models (and beyond), 2025. URL https://arxiv.org/abs/2510.22954.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL https://arxiv.org/abs/2001.08361.

Valentin Khrulkov, Andrey Galichin, Denis Bashkirov, Dmitry Vinichenko, Oleg Travkin, Roman Alferov, Andrey Kuznetsov, and Ivan Oseledets. Gigaevo: An open source optimization framework powered by llms and evolution algorithms, 2025. URL https://arxiv.org/abs/2511.17592.

Jinwoong Kim, Rui Yang, and Huishuai Zhang. Geobuildbench: A benchmark for interactive and executable geometry construction from natural language, 2026. URL https://arxiv.org/abs/2605.13167.

Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06452.

Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution, 2025. URL https://arxiv.org/abs/2509.19349.

Joel Lehman and Kenneth O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011.

Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. Evolution through large models, 2022. URL https://arxiv.org/abs/2206.08896.

Pan Li. Dictionaries, not darwin: Set-level selection beats llm evolution in scientific equation discovery, 2026. URL https://arxiv.org/abs/2607.04108.

Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.02051.

Jiaxu Lou and Yifan Sun. Anchoring bias in large language models: An experimental study, 2024. URL https://arxiv.org/abs/2412.06593.

Simon Malberg, Roman Poletukhin, Carolin M. Schuster, and Georg Groh. A comprehensive evaluation of cognitive biases in llms, 2025. URL https://arxiv.org/abs/2410.15413.

Elliot Meyerson, Mark J. Nelson, Herbie Bradley, Adam Gaier, Arash Moradi, Amy K. Hoover, and Joel Lehman. Language model crossover: Variation through few-shot prompting, 2024. URL https://arxiv.org/abs/2302.12170.

Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

OpenAI. OpenAI o3 and o4-mini system card. https://openai.com/index/o3-o4-mini-system-card/, 2025.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11324.

Asankhaya Sharma. Openevolve: an open-source implementation of AlphaEvolve. GitHub repository, https://github.com/codelion/openevolve, 2025. Repository unpinned; circle-packing example value read at time of access, 2026.

Parshin Shojaee, Iman Mirzadeh, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity, 2025. URL https://arxiv.org/abs/2506.06941.

Kevin Sim, Quentin Renau, and Emma Hart. Beyond the hype: Benchmarking llm-evolved heuristics for bin packing, 2025. URL https://arxiv.org/abs/2501.11411.

Charlie Snell, Eric Wallace, Dan Klein, and Sergey Levine. Predicting emergent capabilities by finetuning, 2024. URL https://arxiv.org/abs/2411.16035.

Chang Su, Zhongkai Hao, Zhizhou Zhang, Zeyu Xia, Youjia Wu, Hang Su, and Jun Zhu. Helix: Evolutionary reinforcement learning for open-ended scientific problem solving, 2026. URL https://arxiv.org/abs/2603.07642.

Maria Thomas, Kristina Gligoric, and Nihar B. Shah. Mitigating llm-based p-hacking by preregistering for the next llm, 2026. URL https://arxiv.org/abs/2606.27687.

Michelle Vaccaro. Preregistration for experiments with ai agents, 2026. URL https://arxiv.org/abs/2606.11217.

Niki van Stein and Thomas Bäck. Llamea: A large language model evolutionary algorithm for automatically generating metaheuristics, 2025. URL https://arxiv.org/abs/2405.20132.

Haorui Wang, Parshin Shojaee, Kazem Meidani, Kunyang Sun, José Miguel Hernández-Lobato, Teresa Head-Gordon, Jiajun He, Chandan K. Reddy, Chao Zhang, and Yuanqi Du. Towards diverse scientific hypothesis search with large language models, 2026. URL https://arxiv.org/abs/2606.10587.

Yiping Wang, Shao-Rong Su, Zhiyuan Zeng, Eva Xu, Liliang Ren, Xinyu Yang, Zeyi Huang, Xuehai He, Luyao Ma, Baolin Peng, Hao Cheng, Pengcheng He, Weizhu Chen, Shuohang Wang, Simon Shaolei Du, and Yelong Shen. Thetaevolve: Test-time learning on open problems, 2025. URL https://arxiv.org/abs/2511.23473.

Marcus Williams, Hannah Sheahan, Cameron Raymond, Tomek Korbak, Deng Pan, Peilin Yang, Leon Maksin, Ningyi Xie, Phillip Guo, Ian Kivlichan, and Micah Carroll. Predicting llm safety before release by simulating deployment, 2026. URL https://arxiv.org/abs/2607.07184.

Xia Yang, Xuanyi Zhang, Hao Hu, and Feng Ji. Beyond accuracy: Evaluating strategy diversity in llm mathematical reasoning, 2026. URL https://arxiv.org/abs/2605.09292.

Dong Zhang. Testing frontier large language models' physics literacy in parallel physical worlds, 2026. URL https://arxiv.org/abs/2607.00276.

Jiayi Zhang, Simon Yu, Derek Chong, Anthony Sicilia, Michael R. Tomz, Christopher D. Manning, and Weiyan Shi. Verbalized sampling: How to mitigate mode collapse and unlock llm diversity, 2026a. URL https://arxiv.org/abs/2510.01171.

Rui Zhang and Zhichao Lu. Rethinking code similarity for automated algorithm design with llms, 2026. URL https://arxiv.org/abs/2603.02787.

Rui Zhang, Fei Liu, Xi Lin, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Understanding the importance of evolutionary search in automated heuristic design with large language models, 2024. URL https://arxiv.org/abs/2407.10873.

Xinhao Zhang, Xi Chen, François Portet, and Maxime Peyrard. What makes an llm a good optimizer? a trajectory analysis of llm-guided evolutionary search, 2026b. URL https://arxiv.org/abs/2604.19440.

Lexin Zhou, Wout Schellaert, Fernando Martínez-Plumed, Yael Moros-Daval, Cèsar Ferri, and José Hernández-Orallo. Larger and more instructable language models become less reliable. *Nature*, 634(8032):61–68, 2024. doi: 10.1038/s41586-024-07930-y.

A The registered arms

Every registered arm, with the section that reports it, its proposer, design, registered test and outcome. Table 2 in §3 is the subset of these rows that carries Table 1.

Table 4: The twenty-one registered arms: section (a letter is an appendix; Sn is a section of the supplement), proposer, design, registered test, outcome. Design gives cells × samples per cell. Registered tests are coded P (prediction), F (falsifier), S (secondary prediction) and R (replication rule), each expanded in its arm's section. The six arms the companion paper reports are named in the final row.

Arm	§	Proposer(s)	Design	Registered test	Outcome
F (square, bare)	3.3, S9	weak tier	7 N × 5–20	P1–P5 exact-value modes	modal at 7/7 N
S (Sonnet direct arm)	3.3	Sonnet tier	3 N × 10	P-S1–P-S4 tier contrasts	1/30 on-prediction, 100%/90% valid (10^{-6}/10^{-9}); one out-of-family escape; 5 of 20 samples at $N = 13/21$ seen before registration, disclosed
O (top alias)	S3	`opus_alias`	3 N × 10	P-O1–P-O4	13% valid; 3 predictions not evaluable (deviation, Table 5)
T (trace)	S6	weak tier	3 N × 20 × 2 arms	P-T1–P-T4 + falsifier	falsifier **triggered** (tie reading); P-T3 fragile
G (rectangle)	S5	16 invocations	2 cells	out-of-sample transfer	partial, 5/11; formula negative result kept
M (extend-cell arm)	3.3, S4	weak tier	4 N × 15	P-M1–P-M4 + F-M1/F-M2	F-M1 **triggered 3/3**: extend branch dies at $m \geq 4$; fifth k confirms; two out-of-family escapes at $N = 20$ (post hoc)
CH (choice)	4.1	weak tier	3 N × 15	P-CH1 vs P-CH2	P-CH1 holds 2/3, P-CH2 1/3 (registered); post-hoc attempt reading: 36/45 reach for the argmax, 30/30 evaluable invalid rows at its order
CC (weak math-only arm)	3.4	weak tier	3 N × 15	P-CC1 vs P-CC2	P-CC1 holds: anchor stays modal; 0/37 above the family argmax
CC2 (replication arm)	3.4	weak tier	3 N × 15	R1 + R2 + F-CC2.1	**REPLICATED**: 0/41 above argmax, anchor modal 3/3
CCS (Sonnet math-only arm)	3.4	Sonnet tier	3 N × 15	P-CCS1 vs P-CCS2	P-CCS2 holds: 0/37 above argmax — the channel, not the tier
CN (held-out N)	4.2, S9.4	weak tier	5 N × 15	P-CN1 vs P-CN2 + S-CN1–3	P-CN1 holds: mode at 4/5 held-out cells, 3/3 discriminating; structure 5/5
CP (perturbed container)	S9.5	weak tier	5 N × 15	P-CP1 vs P-CP2 + S/F	P-CP1 holds: mapped prediction modal 4/5; S-CP1 not met (4/61 rival)
RP (direct recall)	S9.5	weak tier	6 N × 15	P-RP1 vs P-RP2 + F-RP1	P-RP1 holds: 0/44 recalls; 87/87 scored responses UNKNOWN
PP (paraphrase)	S9.5	weak tier	3 rewrites × 2 N × 15	P-PP1 vs P-PP2 + S-PP1/2	P-PP1 holds 6/6: modal under every paraphrase; S-PP1 not met (6/75 rival)
P (pinned path)	3.5, S8	weak tier, bare API	15 cells × 15	P-P1–P-P4 + F-P1	validity collapses: 0/105 valid at the square cells, 4/75 held-out; every direct-emission prediction UNSCOREABLE; P-P3 holds (0 of 12 valid programs clear); same-day agent-runtime control 6/6 valid, anchor 4/6 (post hoc)

Arm	§	Proposer(s)	Design	Registered test	Outcome
CL (library code)	3.5	weak + Sonnet, numpy/scipy	2 tiers × 3 N × 15	P-CL1 vs P-CL2 + F-CL1	weak: 0/11 clear, P-CL1 holds; Sonnet: **23/25 clear at 3/3 cells**, P-CL2 holds, **F-CL1 fires**
CCP (isolating arm)	3.5	Sonnet tier, bare API, math only, 120 s	3 N × 15	P-CCP1 vs P-CCP2 + F-CCP1	P-CCP1 holds: 0/35 clear, none reaches the argmax; best per cell 1.625, 2.1, 2.673 — the library, not the path or the budget
CL-W (weak top-up)	3.5	weak tier, numpy/scipy	3 N × 15, pooled with CL	P-CLW1 vs P-CLW2 + F-CLW1; lenient reading secondary	P-CLW1 holds: 0/19 pooled clear, upper bound 16.8%; lenient reading 4/41, dead zone
B (optimizer alone)	3.5, S7	no model in the loop; a fixed SLSQP program	3 N × 15 seeds, 120 s	P-B1 vs P-B2 + S-B1	P-B1 holds: **45/45 clear**, 3/3 cells; S-B1: beats the Sonnet best at 3/3 cells; amendment 1 (version 1 blocked by the allowlist, kept)
B2 (optimizer alone, larger cells)	3.5 (program in S7)	no model in the loop; arm B's program, restart count changed	primary 2 N × 15 seeds, 120 s; secondary 3 N × 15 seeds, 3600 s	P-B2-1 + S-B2-1 + S-B2-2 + F-B2	P-B2-1 holds: **28/28 valid clear** of 30 launched at $N = 57$ and 59; S-B2-1 holds at 2/3 cells at 50 restarts; S-B2-2 not met at 2/3, $N = 91$ **UNSCOREABLE** (0 valid of 15 launched, all wall-killed); F-B2 not triggered
P-D (runtime component)	3.5, S8	weak tier, bare API, $N = 13$	2 conditions × 15	P-PD1 vs P-PD2 vs P-PD3 + S-PD1	P-PD1 holds (closed at 27/30 rows by amendment, verdict unchangeable): thinking on 12/14 valid, template modal; system line only 0/13 — extended thinking is the component

Six further registered arms — MU (conditioning), L (iterated loop), V (cross-vendor zero-shot), GM (gemini-flash-lite), GM2 (gemma, 4k budget) and GM3 (gemma, 16k budget) — varied a parent, a generation or the vendor. They are reported in the companion paper, which carries their designs, registered tests and outcomes.

B Deviations and disclosure table

Every arm with a registered deviation, an amendment or a post-hoc analysis is listed here, with what is recorded for it. Supplement S10 states each one in full: its registered rule, whether it was followed as registered, and the outcome.

Table 5: Arms with a deviation, an amendment or a post-hoc analysis, and what Supplement S10 records for each.

Arm	**What is recorded**
F, S and O (direct emission)	arm S's registration disclosed samples seen before it; three arm-O predictions declared not evaluable post hoc; the ladder inclusion and the cap exclusion post hoc
T (trace)	the registered falsifier reported under both readings; the wave stratification post hoc
M (extend cell)	the registered rejection rule applied and the excluded invocations counted; the out-of-family sweep post hoc
CH (choice)	the attempt-level reading post hoc
CC2 and CCS (registered follow-ups)	registered after arm CC's results were known and disclosed as such; one arm-CC2 invocation used tools, scored as emitted
CN (held-out N)	the ledger transcription disclosed and the reconstructed rows discarded

Arm	What is recorded
CP, RP and PP (container, recall, paraphrase)	registered before sampling; the recall arm's control amendment; the paraphrases authored results-aware, disclosed
P (pinned path)	registered before sampling, the motivation post-review; the temperature and runtime-control diagnostics post hoc
P-D (runtime component)	registered before sampling; the run closed by amendment before its last rows
B and B2 (optimizer alone)	arm B's amendment 1, its blocked first version kept in the artifact; arm B2's worker-count change in the secondary reading
CL, CL-W and CCP (library, top-up, isolating)	registered before sampling; arm CL's execution-mechanism correction; the lenient reparse post hoc
Pooled diagnostics	the uniform-template null, the structural k back-out and the ambition diagnostic, all post hoc

C Provenance, in full

Section 7 summarises this appendix; here every registration, commit, ledger and script is in one place.

- **Registration.** Prompts were hashed with SHA-256 and the hashes recorded before sampling, with the predicted values to be tested. Bare square arm (F): the five exact-value predictions in the header of `arm_f_repro.py`. Sonnet direct arm (S) and top-alias arm (O): `arm_s_preregistration.txt` and `arm_o_preregistration.txt`. Trace arm (T): `arm_t_preregistration.txt`, hash `ab7900a8...`. Extend-cell arm (M): `arm_m_preregistration.txt` with `arm_m_prompts.json`, committed at `e528c6b` before sampling. Conditioning arm (MU): `arm_mu_preregistration.txt` with `arm_mu_prompts.json`, committed at `2b7d202` before sampling; decision thresholds, power analysis, tie conventions and falsifier all registered. Second-vendor chain (GM, GM2, GM3): `arm_gm_preregistration.txt` at commit `37b3adb`, `arm_gm2_preregistration.txt` at `3019aab`, `arm_gm3_preregistration.txt` at `c0b5b97`. The gemma arm's (GM3) serving-path deviation, `arm_gm3_amendment1_openrouter.md` at `67cc4a1`, was registered before the affected cells were sampled. Cross-vendor arm (V): `arm_v_preregistration.md` at commit `5444f10`, ancestry-checked by its runners before any sampling; three amendments, each committed before the sampling it governs. First weak math-only arm (CC): `arm_cc_preregistration.txt` with frozen prompts, executor and smoke test, committed and pushed to the public remote at `285deca` before its first invocation. Replication arm (CC2) and Sonnet math-only arm (CCS): registered together in `arm_cc2_preregistration.txt` at commit `4d7b7c5`, pushed before sampling. That registration was written after the first weak math-only arm's results were known, and says so. Held-out-cells arm (CN): `arm_cn_preregistration.txt` at commit `69a1642`, pushed before sampling, registered after the rest of the manuscript was complete. Its motivation is post hoc with respect to the paper's results, the registration says so, and its predictions were fixed before its first invocation. Container arm (CP) and recall arm (RP): `arm_cp_preregistration.txt` and `arm_rp_preregistration.txt`, each with its prompt-hash JSON and analysis script, registered together at commit `105e8b7`, pushed before the first invocation. The recall arm's one cell change before registration, $N = 32$ to $N = 30$, and its positive-control amendment at `15555eb` are documented in its registration. Paraphrase arm (PP): `arm_pp_preregistration.txt`, prompt hashes and scorer committed at `2e89614`, pushed before the first invocation; it discloses that its paraphrases were authored results-aware. The rectangle arm's (G) transfer was registered before collection and never refitted. Pinned-path arm (P) and library arm (CL): registrations, builders, frozen prompts, runners and analysis scripts, listed below, committed and pushed together at `aa473ad` before either was sampled. On multiplicity: twenty-one registered arms, and no correction across them. Each arm's predictions were registered with its own bars and falsifiers before its sampling, with the arm-S exception the deviations table records, and every arm is reported regardless of outcome. Three second-vendor instruments failed or stalled (arms GM and GM2, and the gemma arm's first-party path) before the gemma arm (GM3) completed on its amended path, the falsifier of Supplement S4 fired, and no arm was selected for inclusion on its result. The protection against the garden of forking paths is registration and exhaustive reporting, not a family-wise error rate. A reader who wants one can count the twenty-one arms in Table 4, whose last row names the six the companion paper reports. That defence covers the registered predictions, not the post-hoc diagnostics: which unregistered analyses we ran is a degree of freedom registration does

not constrain. Table 5 lists them. Three cut against us: the out-of-family sweep of Supplement S4, the rival-structure check of the companion paper, and the wave stratification of Supplement S6.4.

- **Four limits on that claim.** (i) The digests cover the task prompt only. The subagent inherits instruction files outside it, so what is locked is a *fragment* of the conditioning context. A replicator cannot reconstruct those files, and we cannot establish that they were identical across the bare-square and trace windows, the boundary Supplement S6.4 identifies as a confound. (ii) `arm_f_prompts.json` covers $N = 13$, 17, 31, 35 and 37; the $N = 21$ bare hash is registered in `arm_t_preregistration.txt`. The $N = 43$ **bare hash appeared in no preregistration and no prompts file** until the pinned-path arm's, existing only in the ledger. It recomputes exactly from the bare template, but recomputation after the fact is not registration. (iii) **No hash was ever registered for the pilot prompt**, whose drift the registration describes only in prose; the ten pilot rows carry `prompt_sha256: null` in the corrected ledger with an explanatory note. (iv) The rectangle ledger, `arm_g_candidates.jsonl`, carries no prompt hashes, no run dates and no proposer fields.
- **Ledger metadata correction.** The shipped ledger's provenance fields contradicted the paper in three places. Trace rows carried the *bare* prompt hash for their cell; `run_date` was uniformly `2026-07-30` on all 215 rows; the Sonnet and `opus_alias` rows were stamped with the Haiku alias and dated id. All three are corrected in `arm_f_candidates_v2.jsonl`, metadata only. Every scored quantity, geometry, raw output, arm label and sample id is byte-identical to v1, verified field by field. Each corrected field carries a sibling `*_v1` field. `arm_f_candidates.jsonl` is unmodified, sha256 `02ecabcd...`; v2 is `9f845f78...`. Correction rules, sources, wave-boundary arithmetic and caveats, including that `run_date` has day granularity only, are in **`ledger_v2_corrections.md`**. Analysis scripts read v2.
- **Excluded and unreleased records.** The five concurrency-cap rejections appear in the ledger in no form, no `excluded_reason` field exists, and the exclusion rule was not registered; both rates are reported in Supplement S9.3. The `opus_alias` latency and token-count anomalies of Supplement S3 come from the runtime session transcript; no latency or token field exists in any released artifact.
- **Analysis artifacts.** Raw outputs are stored verbatim and scoring is deterministic and local. Bare square: `arm_f_raw.json`, `arm_f_candidates.jsonl`, `arm_f_candidates_v2.jsonl`. Rectangle: `arm_g_candidates.jsonl`. Extend cell: `arm_m_collect.jsonl`, scored by `arm_m_analysis.py`. Conditioning: `arm_mu_collect.jsonl`, scored by `arm_mu_analysis.py`, frozen output `arm_mu_results.txt`. Gemma: `arm_gm_gm3_checkpoint.jsonl`, every row's raw API response verbatim with serving path and provider tagged, scored by `arm_gm3_analysis.py` into `arm_gm3_report.json` with both serving-path accountings. Value claims are gated on an independent LP oracle, not on the closed form being tested. The oracle, `n_sweep_forecast.py` with `verify_against_lp`, checks 83 configurations at $k = 2 \ldots 7$ to within 10^{-9} (the further 130 at $k = 8$ and 9 are listed in Supplement S9.1) and produced the negative result of Supplement S5.2. For the trace arm, Supplement S6, the faithfulness rubric `p7_faithfulness_rubric.md` was frozen by git commit `e181d2a` on 2026-08-01 at 03:56:56, before any rescoring. All 60 blind labels with justifications are in `p7_blind_labels.json`. Also included: `arm_t_analysis.py`, `n_sweep_forecast.json` and `rect_forecast.json` for the trace and rectangle arms, Supplement S6 and S5; `p11_mode_baseline.json`; `STATE.md`. Every table regenerates from raw outputs by a command documented in `HOW_TO_RUN.md`.
- **Internal checks.** Before submission the manuscript passed an internal adversarial check. Language models of other families, `deepseek-v4-pro`, `deepseek-v4-flash` and Gemini, under written protocols, independently recomputed the paper's statistics from the released ledger. Thirty-two claims were corrected as a result, itemized in the supplementary `corrections_ledger.md`. This is quality assurance, not peer review, and carries no external authority.

C.1 Per-arm ledgers and scripts

One entry per arm or registered group: ledgers, scripts, reports and the running corpus count. Letters stay in the headings so an auditor can join each entry to its row in the arm table of Appendix A.

- **The conditioning arm and the argmax-in-hand arm (MU, CH).** Raw rows verbatim in `arm_mu_collect.jsonl`: 180 of 180 launched invocations collected across the two arms, no runtime deaths. Scoring `arm_mu_analysis.py`, committed before sampling completed; frozen output `arm_mu_results.txt`. These arms bring the study corpus to 468 invocations.
- **The first weak math-only arm (CC).** Raw rows verbatim in `arm_cc_collect.jsonl`: 45 of 45 launched invocations collected, no runtime deaths. Scoring `arm_cc_analysis.py`; frozen report `arm_cc_report.json`; summary `arm_cc_results.txt`.
- **The pinned-path arm (P) and the thinking-budget arm (P-D).** Pinned-path arm: registration `arm_p_preregistration.txt`; prompts and hashes `arm_p_prompts.json`; runner `arm_p_run.py`; scorer `arm_p_analysis.py`; report `arm_p_report.json`. Ledger `arm_p_collect.jsonl`: 225 rows, six refused by the serving path. Post-hoc diagnostics `arm_p_diag_temperature.jsonl` and `arm_p_diag_runtime.jsonl`, labelled post hoc in their headers. Thinking-budget arm: registration `arm_pd_preregistration.txt` at commit `5554cc0`; builder `arm_pd_build.py`, the pinned-path arm's $N = 13$ prompt with hash asserted; spec `arm_pd_prompts.json`; runner `arm_pd_run.py`. Scorer `arm_pd_analysis.py`, the pinned-path scorer imported unmodified. Ledger `arm_pd_collect.jsonl`: 27 rows plus the HTTP 402 refusal rows kept for resume. Report `arm_pd_report.json`, marked INTERIM.
- **The library arm (CL).** Registration `arm_cl_preregistration.txt`; builder `arm_cl_build.py`, one clause replaced in the weak math-only arms' prompt, hash asserted; prompts `arm_cl_prompts.json`; runner `arm_cl_run.py`. Scorer `arm_cl_analysis.py`, with the driver and the `numpy` and `scipy` versions recorded in the report. Ledger `arm_cl_collect.jsonl`: 90 rows. Report `arm_cl_report.json`. The post-hoc re-execution and lenient reparse are `recount_cl.py` and `cl_lenient_reparse.py` in the paper repository's `loop/` directory, outputs `cl_recount.json` and `cl_lenient.json`.
- **The isolating arm (CCP).** Registration `arm_ccp_preregistration.txt` at commit `ebf213e`; builder `arm_ccp_build.py`, the weak math-only arms' prompts byte-identical, hashes asserted; prompts `arm_ccp_prompts.json`. Runner `arm_ccp_run.py`, with the one-worker amendment after the row-24 interruption at `32fd3c7`. Scorer `arm_ccp_analysis.py`, the library arm's pipeline with the allowlist set to `math`. Ledger `arm_ccp_collect.jsonl`: 45 rows plus the one HTTP 402 refusal row, both segments timestamped. Report `arm_ccp_report.json`.
- **The weak top-up (CL-W).** Registration `arm_clw_preregistration.txt` at commit `f73222a`; builder `arm_clw_build.py`, the weak library arm's prompts byte-identical, hashes asserted; prompts `arm_clw_prompts.json`; runner `arm_clw_run.py`. Scorer `arm_clw_analysis.py`, the library arm's pipeline over the pooled 90 rows, both readings per row. Ledger `arm_clw_collect.jsonl`: 45 rows. Report `arm_clw_report.json`, every row's registered and lenient result.
- **The optimizer-alone baseline (B).** Registration `arm_b_preregistration.txt` at commit `1932cc0`; amendment `arm_b_amendment1_fixed_restarts.md` at `c486843`; program `arm_b_baseline.py`, sha asserted by the runner. Runner `arm_b_run.py`, the library arm's pipeline with no model. Ledger `arm_b_collect.jsonl`: 45 rows, each the fixed program with N and seed substituted. Report `arm_b_report.json`. The run the allowlist blocked is kept as `arm_b_v1_blocked_collect.jsonl` with its report and log. No model was invoked; the study corpus stays at 1500 model invocations (running total in Supplement S6.1).
- **The optimizer-alone baseline at larger cells (B2).** Registration `arm_b2_preregistration.txt` at commit `00a1b69`; runner `arm_b2_run.py`, both readings; program `arm_b_baseline.py` unchanged except for its restart count, template sha asserted before substitution. Ledgers `arm_b2_collect_primary.jsonl` and `arm_b2_collect_secondary.jsonl`, reports `arm_b2_report_primary.json` and `arm_b2_report_secondary.json`, split per reading where the registration named one ledger and one report. Bridge row `arm_b_bridge_run.py` with `arm_b_bridge_collect.jsonl` and `arm_b_bridge_report.json`. The secondary's stopped first launch is kept as `arm_b2_secondary_launch1_4workers.log`. The machine is a 22-core host, not

arm B's, and each report records the worker count its reading ran at. No model was invoked; the study corpus stays at 1500 model invocations (running total in Supplement S6.1).

- **The replication arm (CC2) and the Sonnet math-only arm (CCS).** Ledgers `arm_cc2_collect.jsonl` and `arm_ccs_collect.jsonl`, 45 of 45 each, no runtime deaths. Runner `arm_cc2_analysis.py`, the first weak math-only arm's registered pipeline unmodified. These arms bring the study corpus to 603 invocations (the running total when they ran; entries here are not in chronological order).